

Cơ chế huấn luyện, từ vi mô tới vĩ mô (Training Mechanics, from Micro to Macro)

PHỤ THUỘC A/01-toan-toi-thieu (quy tắc chuỗi, phương sai, số điều kiện); B/09-thiet-ke-ham-mat-mat (biết mất mát là gì trước khi hỏi cách tối thiểu hoá nó). Chương này là tiền đề cho toàn bộ Phần C: mọi kiến trúc ở các chương sau đều được huấn luyện bằng bộ máy mô tả ở đây.

TÓM TẮT

Cả chương là câu trả lời cho *một* câu hỏi: làm sao đưa **gradient** đi qua một mạng sâu mà không làm nó chết hoặc nổ. Gần như mọi thủ thuật trong một file cấu hình huấn luyện — từ cách khởi tạo, chọn hàm kích hoạt, đặt normalization, tới warmup và **mixed precision** — tồn tại vì lý do đó. Đọc xong bốn mục, bạn tự viết được lượt ngược cho một tầng mới và đặt được siêu tham số có căn cứ. Bạn cũng chẩn đoán được một lần chạy hỏng trong vài phút, và đưa được mô hình lên nhiều GPU đúng thứ tự. Nếu chỉ nhớ một điều: đây là chương duy nhất trong cây mà kỹ năng chỉ hình thành qua tay — đọc hiểu hết mà chưa từng cố ý làm hỏng một lần chạy thì coi như chưa học.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán; “huấn luyện” nghĩa là đi tìm bộ tham số làm dự đoán sát nhất.
- **tham số (parameter, weight)** — các con số bên trong mô hình mà huấn luyện được phép thay đổi; một mạng thị giác cỡ vừa có hàng chục triệu con số như vậy.
- **hàm mất mát (loss function)** — hàm biến dự đoán và nhãn thành một số duy nhất đo mức sai; toàn bộ huấn luyện là bài toán làm giảm số đó.
- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm mất mát TĂNG nhanh nhất, nên ta đi ngược lại.
- **batch và epoch** — “batch” là nhóm mẫu đi qua mạng cùng lúc; một “epoch” là khi mô hình đã đi hết tập huấn luyện một lượt. Một epoch gồm nhiều bước cập nhật, mỗi bước một batch.
- **learning rate** — độ dài bước mà thuật toán tối ưu đi theo hướng ngược gradient ở mỗi lần cập nhật; đây là siêu tham số quan trọng nhất trong cả chương.
- **kích hoạt và hàm kích hoạt** — “kích hoạt” là giá trị đầu ra của một tầng giữa mạng; “hàm kích hoạt” là phép phi tuyến sinh ra nó, ví dụ ReLU. Tên gần giống nhưng là hai thứ khác nhau.
- **normalization** — phép chuẩn hoá lại phân bố của kích hoạt ở giữa mạng về trung bình 0 độ lệch chuẩn 1; nó tồn tại để giữ tín hiệu không trôi khi đi qua nhiều tầng.
- **overfitting** — mô hình khớp rất tốt dữ liệu huấn luyện nhưng kém trên dữ liệu mới; đây là chủ đề của *mục 4*.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao một mạng 10^9 tham số huấn luyện được trong vài ngày, trong khi vi phân thuận cần 10^9 lượt chạy để có cùng lượng thông tin?
- Q2. Nếu gradient chỉ là quy tắc chuỗi — thứ đã có từ thế kỷ XVII — vì sao suốt hai mươi năm người ta không huấn luyện nổi mạng sâu?

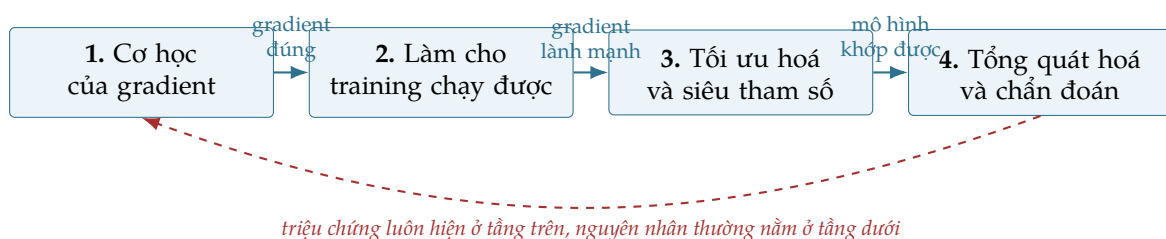
- Q3.** Bầy *optimizer* quen thuộc thường được dạy như bầy lựa chọn trong một menu. Vì sao thực ra chúng là bầy lần vá liên tiếp, và cái nào sinh ra chỉ để sửa lỗi do chính cái trước tạo ra?
- Q4.** Một mạng đủ sức học thuộc cả một tập ảnh với nhãn xáo ngẫu nhiên — vậy vì sao nó vẫn tổng quát hoá tốt khi nhãn là chân trị?
- Q5.** *bf16* có dải động rộng đúng bằng *fp32*. Vậy vì sao huấn luyện ở *bf16* vẫn buộc phải giữ một bản trọng số *fp32*?

1.1 Vì sao chương này chia làm bốn mục

Bốn mục dưới đây không phải bốn chủ đề song song. Chúng là bốn tầng của cùng một câu hỏi, xếp từ vi mô tới vĩ mô. Và *mỗi tầng chỉ có nghĩa khi tầng dưới nó đã đúng*. Đó là lý do thứ tự đọc ở đây là bắt buộc chứ không phải gợi ý.

- 1. Cơ học của gradient** hỏi: gradient được tính thế nào, và vì sao cách tính đó rẻ. Đây là tầng vi mô nhất — một phép toán, một tầng, một đồ thị. Không nắm tầng này thì ba tầng sau chỉ là các quy tắc phải học thuộc.
- 2. Làm cho training chạy được** hỏi: gradient đúng rồi thì làm sao nó sống sót qua năm mươi tầng. Ở đây xuất hiện đại lượng trung tâm của cả chương — hệ số nhân của tích các Jacobian — và ba hướng tấn công vào nó.
- 3. Tối ưu hoá và siêu tham số** giả định gradient đã lành mạnh và hỏi tiếp: nên đi bước dài bao nhiêu, theo hướng nào, ở độ chính xác số nào. Đây là tầng mà bầy *optimizer* quen thuộc hiện ra như một chuỗi vá lỗi chứ không như một menu.
- 4. Tổng quát hoá và chẩn đoán** là tầng vĩ mô nhất: mô hình chạy tốt trên dữ liệu huấn luyện rồi, còn ngoài đời thì sao, và khi mọi thứ không như kỳ vọng thì tìm nguyên nhân theo trình tự nào.

Hình 1.1 cho thấy vì sao thứ tự này không đảo được. Mỗi tầng tiêu thụ đúng thứ mà tầng dưới sản xuất ra. Một lỗi ở tầng dưới lại luôn hiện ra thành triệu chứng ở tầng trên. Đó chính là lý do người ta hay tune learning rate để chữa một bug ở lượt ngược.



Hình 1.1. Bốn tầng của chương, và chiều mà lỗi lan truyền. Mũi tên liền là quan hệ “tầng sau giả định tầng trước đã đúng”; mũi tên đứt là điều khiến việc gỡ lỗi khó: một bug ở tầng 1 hầu như luôn biểu hiện thành một đường cong xấu ở tầng 4.

Bản đồ chương 12 — Cơ chế huấn luyện

TÓM TẮT

Chương 12 gồm bốn mục và chúng *không* song song: đây là bốn tầng của cùng một câu hỏi, xếp từ vi mô tới vĩ mô, mỗi tầng giả định tầng dưới đã đúng. Đọc theo đúng thứ tự $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$; đọc nhảy cóc thì mục 3 và mục 4 biến thành danh sách quy tắc phải học thuộc. Nếu chỉ nhớ một điều: câu hỏi trung tâm của cả chương là *làm sao đưa gradient đi qua một mạng sâu mà không làm nó chết hoặc nổ*. Mọi kỹ thuật trong chương là một câu trả lời cho đúng câu hỏi đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm mất mát TĂNG nhanh nhất, nên ta đi ngược lại.
- **lan truyền ngược (backpropagation)** — thuật toán tính gradient bằng cách đi ngược đồ thị phép tính một lượt duy nhất; nội dung của mục 1.
- **siêu tham số (hyperparameter)** — con số do người đặt trước khi huấn luyện chứ không học từ dữ liệu — learning rate, batch size, cường độ weight decay; nội dung của mục 3.
- **tổng quát hoá (generalization)** — khả năng mô hình chạy tốt trên dữ liệu chưa từng thấy, chứ không chỉ trên dữ liệu đã huấn luyện; nội dung của mục 4.

Có gì trong chương này

Bảng 1.1. Bốn mục của chương 12: câu hỏi trung tâm, phụ thuộc, và mức độ đầu tư.

Mục	Câu hỏi trung tâm	Phụ thuộc	Mức độ
1. Cơ học của gradient	Gradient được tính thế nào, vì sao cách đó rẻ, và nó tồn bộ nhớ ở đâu?	A/01	Cốt lõi [T1]
2. Làm cho training chạy được	Gradient đúng rồi thì làm sao nó sống sót qua năm mươi tầng?	Mục 1	Cốt lõi [T1]
3. Tối ưu hoá và siêu tham số	Đi bước dài bao nhiêu, theo hướng nào, ở độ chính xác số nào?	Mục 2	Cốt lõi [T2]
4. Tổng quát hoá và chẩn đoán	Mô hình khớp được dữ liệu rồi, còn ngoài đời thì sao — và hỏng thì tìm ở đâu?	Mục 3, B/07	Cốt lõi [T2]

Vì sao lại chia như vậy

Cách chia này theo *độ trừu tượng của đối tượng được nói tới*, chứ không theo thứ tự lịch sử hay theo mức độ phổ biến. Mục 1 nói về một phép toán và một đồ thị; mục 2 nói về một mạng; mục 3 nói về một quá trình huấn luyện; mục 4 nói về quan hệ giữa quá trình đó và thế giới bên ngoài. Lợi ích của cách chia này là mỗi mục có một đại lượng trung tâm rõ ràng. Lần lượt: tích vector-Jacobian, hệ số nhân của tích các Jacobian, độ dài bước đi, và khoảng cách train-val. Gặp một kỹ thuật lạ, bạn biết ngay nó thuộc tầng nào.

Cách chia này cũng làm lộ ra một quan hệ mà cách chia theo chủ đề che mất: *triệu chứng luôn xuất hiện ở tầng trên, nguyên nhân thường nằm ở tầng dưới*. Một lỗi ở lượt ngược (tầng 1) biểu hiện thành một đường cong học xấu (tầng 4). Đây là lý do mục 4 dành phần lớn dung lượng cho một cây chẩn đoán đi từ trên xuống. Thấy một đường cong lạ thì phản xạ đúng là đi xuống, không đi ngang.

Đọc theo thứ tự nào

1. Nếu bạn mới bắt đầu: đọc tuần tự $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$, và làm mini project của chương *ngay sau mục 4* chứ không để dành — nó là chỗ kiến thức chuyển thành phản xạ.
2. Nếu bạn đã huấn luyện mạng vài lần và muốn lấp lỗ hổng: đọc mục 3 và mục 4 trước, quay lại mục 1 khi gặp câu hỏi về bộ nhớ hoặc khi phải viết một tầng mới.
3. Nếu bạn đang gỡ một lần chạy hỏng *ngay bây giờ*: vào thẳng cây chẩn đoán và bảng triệu chứng ở mục 4, rồi đi ngược lên tầng mà cây chỉ tới.

Cốt lõi và tham khảo

Cả bốn mục đều thuộc phần cốt lõi của cây, đây là một trong ít chương như vậy. Trong nội bộ chương, ba phần đáng quay lại nhiều lần là: ba mẫu gradient cộng quy tắc rẽ nhánh (mục 1), lập luận phương sai dẫn tới khởi tạo He (mục 2), và mạch bảy bước của optimizer (mục 3). Ba phần có thể đọc lướt lần đầu và tra lại khi cần là: chi tiết công thức lượt ngược của BatchNorm, bảng bốn biến thể normalization, và bảng bốn mức song song hoá dữ liệu.

Nối vào phần còn lại của cây

Chương này tiêu thụ A/01-toan-toi-thieu và B/09-thiet-ke-ham-mat-mat; nó cấp đầu vào cho *tần bộ* Phần C, vì mọi kiến trúc ở các chương sau đều được huấn luyện bằng bộ máy này. Ba nhánh nối ngang đáng nhớ: chi phí bộ nhớ kích hoạt nối sang B/11-gioi-han-tinh-toan; dải động của fp16 và bf16 nối sang D/20-toi-uu-va-nen-mo-hinh; và nhận xét “phần lớn cải tiến kiến trúc hoá ra là cải tiến công thức huấn luyện” nối sang E/24-thi-nghiem-va-ablation.

1.2 Cơ học của gradient (Mechanics of the Gradient)

PHỤ THUỘC A/01-toan-toi-thieu (quy tắc chuỗi, Jacobian, chuẩn ma trận); hữu ích nhưng không bắt buộc: B/11-gioi-han-tinh-toan nếu muốn hiểu vì sao chi phí bộ nhớ ở cuối mục này lại là ràng buộc thật chứ không phải chi tiết phụ.

TÓM TẮT

Mục này giải thích vì sao thứ tự áp dụng quy tắc chuỗi quyết định chi phí huấn luyện, và vì sao **lan truyền ngược** (*backpropagation*) rẻ hơn vì phân thuận khoảng tám bậc độ lớn trên một mạng thật. Đọc xong bạn tự viết được lượt ngược cho một tầng bất kỳ. Bạn cũng tự bắt được lỗi của nó bằng kiểm tra gradient số, và ước lượng được bộ nhớ mà kiến trúc sẽ ngốn. Nếu chỉ nhớ một điều: lan truyền ngược không phải công thức riêng của mạng nơ-ron, nó là thuật toán vi phân cho mọi đồ thị khả vi, nên mọi kiến trúc lạ chỉ còn là một đồ thị mới.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán; “huấn luyện” nghĩa là đi tìm bộ tham số làm dự đoán sát nhãn nhất.
- **tham số (parameter, weight)** — các con số bên trong mô hình mà quá trình huấn luyện được phép thay đổi; một mạng thị giác cỡ vừa có hàng chục triệu con số như vậy.
- **bias** — **phân biệt hai nghĩa** — trong công thức $Wx + b$, b là *tham số cộng* của một tầng. Từ “bias” trong thống kê lại nghĩa là *độ lệch hệ thống* của một bộ ước lượng. Mục này chỉ dùng nghĩa thứ nhất.

- **hàm mất mát (loss function)** — hàm biến dự đoán và nhả thành một số duy nhất đo mức sai; mọi thứ trong huấn luyện đều nhằm giảm số đó.
- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm mất mát TĂNG nhanh nhất, nên ta đi ngược lại.
- **Jacobian** — ma trận đạo hàm riêng của một hàm nhiều đầu vào, nhiều đầu ra; nó là dạng tổng quát của gradient và là đối tượng chính mà lượt ngược thao tác.
- **tensor** — mảng số nhiều chiều — một ảnh RGB là tensor ba chiều, một lô ảnh là tensor bốn chiều; đây là đơn vị dữ liệu mà mọi phép toán trong mạng làm việc với.
- **batch (lô)** — nhóm mẫu được đưa qua mạng cùng lúc để tận dụng phần cứng song song; kích thước lô ảnh hưởng tới cả tốc độ lẫn chất lượng, và ta sẽ thấy vì sao ở các mục sau.
- **kích hoạt (activation)** — giá trị đầu ra của một tầng ở giữa mạng, tức dữ liệu đang trên đường đi; đừng nhầm với “hàm kích hoạt” là phép phi tuyến sinh ra nó.

A. Đặt vấn đề

Một mạng hiện đại là hàm hợp của vài nghìn phép toán sơ cấp. Nó phụ thuộc vào 10^7 đến 10^9 tham số vô hướng. Mỗi bước huấn luyện lại cần đạo hàm của *một* số duy nhất — giá trị **hàm mất mát (loss function)** — theo *toàn bộ* chỗ tham số đó. Câu hỏi không phải “quy tắc chuỗi có áp dụng được không”, vì nó luôn áp dụng được. Câu hỏi là: áp dụng quy tắc chuỗi *theo thứ tự nào* để chi phí là một lượt đi thay vì một trăm triệu lượt. Toàn bộ nội dung của **lan truyền ngược (backpropagation)** nằm trong câu hỏi thứ tự đó, chứ không nằm trong bản thân đạo hàm.

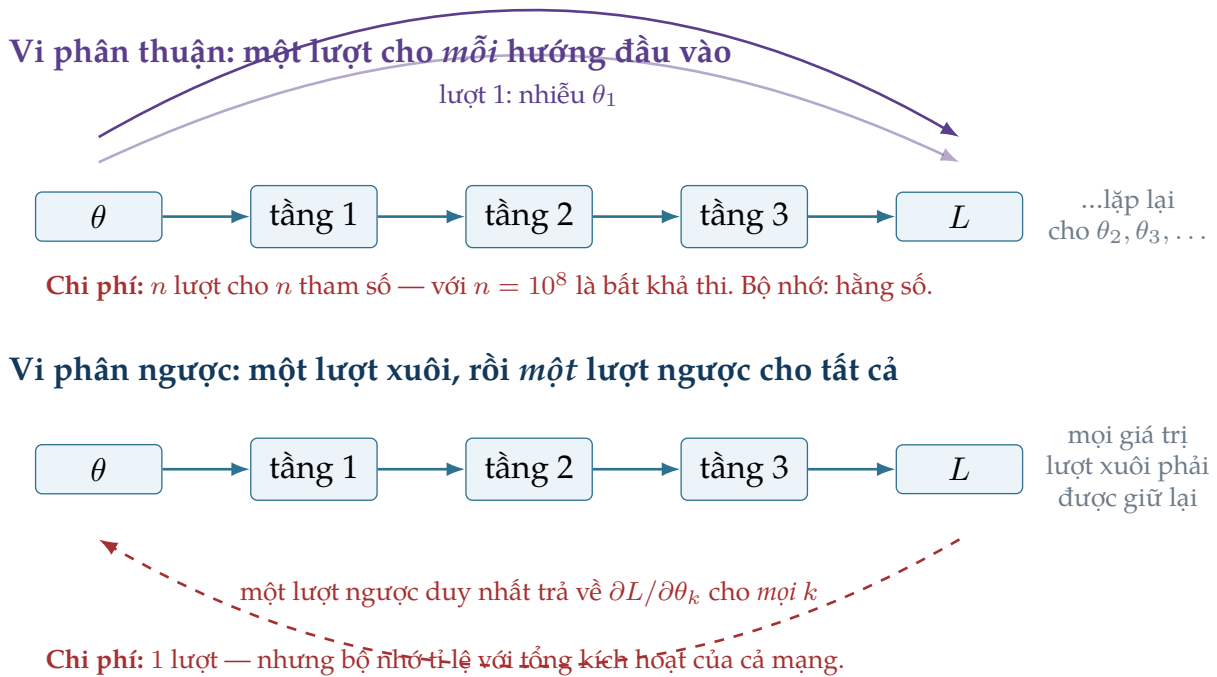
Bạn đã gặp mọi nguyên liệu của bài toán này ở phía tối ưu hoá phi tuyến trong SLAM: dựng Jacobian, nhân dồn các Jacobian cục bộ, giải hệ chuẩn tắc. Nhưng hình dạng bài toán ở đó ngược hẳn. Trong bundle adjustment, số dư (*residual*) rất nhiều còn tham số tương đối ít, nên việc dựng tường minh ma trận J rồi làm việc với $J^T J$ là hợp lý. Trong huấn luyện mạng, đầu ra chỉ có *một* chiều còn đầu vào có hàng trăm triệu chiều. **Forward-mode autodiff** lan truyền một đạo hàm theo hướng: mỗi lượt cho bạn đạo hàm theo đúng một hướng đầu vào, nên cần 10^8 lượt. **Reverse-mode autodiff** lan truyền độ nhạy ngược từ đầu ra: mỗi lượt cho bạn đạo hàm theo *tất cả* đầu vào, nên cần đúng một lượt cho mỗi đầu ra vô hướng. Tỉ số 10^8 giữa hai cách là lý do tồn tại của lan truyền ngược [1]. Cùng một quy tắc chuỗi, hai thứ tự áp dụng, chi phí lệch nhau tám bậc độ lớn.

Mục này cho bạn ba việc cụ thể. Bạn tự viết được lượt ngược cho một tầng chưa có trong thư viện. Bạn tự bắt được lỗi của nó bằng kiểm tra số. Và bạn ước lượng được bộ nhớ mà một kiến trúc sẽ ngốn. Trước đó cả ba đều phải tin vào framework.

Bảng 1.2. Hai chế độ vi phân tự động. Chúng dùng đúng cùng một quy tắc chuỗi và chỉ khác nhau ở thứ tự nhân các Jacobian, nhưng thứ tự đó quyết định toàn bộ chi phí.

Chế độ	Cơ chế	Chi phí cho n đầu vào, m đầu ra	Hỏng / không dùng khi
Forward-mode	Lan truyền đạo hàm theo một hướng đầu vào, cùng chiều lượt xuôi	n lượt; bộ nhớ hằng số	n lớn — huấn luyện mạng có 10^8 tham số là bất khả thi
Reverse-mode	Lan truyền độ nhạy của một đầu ra, ngược chiều lượt xuôi	m lượt; bộ nhớ tỉ lệ tổng kích hoạt	m lớn (cần cả Jacobian đầy đủ), hoặc bộ nhớ không đủ giữ kích hoạt

Bảng 1.2 nói gọn lại: huấn luyện là bài toán “một đầu ra, rất nhiều đầu vào”, và đó chính xác là hình dạng mà reverse-mode được sinh ra để phục vụ.



Hình 1.2. Cùng một quy tắc chuỗi, hai thứ tự nhân, hai chi phí lệch nhau tám bậc độ lớn. Vi phân thuận đi cùng chiều lượt xuôi nên mỗi lượt chỉ trả lời được cho một hướng đầu vào; vi phân ngược đi ngược lại nên một lượt phủ mọi tham số. Điểm đáng nhớ nằm ở dòng cuối mỗi nửa hình: cái ta tiết kiệm được ở thời gian phải trả lại bằng bộ nhớ.

Hình 1.2 cho thấy điều mà bảng không nói được: hai chế độ dùng cùng một đồ thị và cùng các Jacobian cục bộ, chỉ khác chiều duyệt — và chính chiều duyệt sinh ra cả lợi thế lẫn cái giá.

Nhận thức then chốt cần mang ra khỏi mục này là: lan truyền ngược không phải công thức riêng của mạng nơ-ron. Nó là thuật toán chung cho mọi đồ thị có hướng không chu trình mà mỗi nút khả vi. Khi bạn đã tin điều đó, một kiến trúc lạ không còn là một thứ phải học thuộc; nó chỉ là một đồ thị mới, và bạn đã biết cách vi phân mọi đồ thị.

TRỰC GIÁC

Hãy nghĩ về forward pass như một dây chuyền sản xuất: nguyên liệu đi qua ba mươi trạm, mỗi trạm biến đổi một chút, cuối dây chuyền ra một sản phẩm có một con số sai lệch so với chuẩn. Forward-mode giống việc rung nhẹ một nút vặn rồi chạy lại cả dây chuyền để xem sai lệch đổi bao nhiêu. Cách đó đúng, nhưng phải lặp lại đúng bằng số nút vặn. Reverse-mode làm ngược: bắt đầu từ sản phẩm lỗi, hỏi trạm cuối “một đơn vị sai ở đây tương ứng với bao nhiêu sai ở đầu vào của anh”, rồi chuyển đúng câu hỏi đó ngược lên trạm trước. Một lượt đi ngược duy nhất trả lời cho mọi nút vặn trên toàn dây chuyền, vì mỗi trạm chỉ cần biết quy tắc quy đổi cục bộ của riêng nó.

B. Đồ thị tính toán và tích vector-Jacobian (computational graph, VJP)

Hình thức hoá của ẩn dụ trên rất gọn. Biểu diễn phép tính thành một đồ thị: nút là giá trị trung gian, cạnh là phụ thuộc. Lượt xuôi duyệt đồ thị theo thứ tự tô-pô và tính giá trị; lượt ngược duyệt theo thứ tự tô-pô đảo và tính *adjoint* $\bar{v} = \partial L / \partial v$ cho từng nút. Mỗi nút chỉ cần biết một thứ: cách biến adjoint ở đầu ra của nó thành adjoint ở đầu vào của nó. Nếu nút thực hiện $y = g(x)$ thì quy

tắc đó là

$$\bar{x} = \left(\frac{\partial y}{\partial x} \right)^\top \bar{y},$$

tức là một **tích vector-Jacobian** (*vector-Jacobian product, VJP*). Điểm quan trọng về mặt kỹ thuật là ta *không bao giờ* dựng ma trận Jacobian ở giữa. Xét một tầng affine $y = Wx + b$ với $W \in \mathbb{R}^{4096 \times 4096}$. Jacobian theo W là ma trận $4096 \times 16,8$ triệu, khoảng $6,9 \cdot 10^{10}$ phần tử. Không thể lưu nổi. Nhưng VJP của nó chỉ là ba biểu thức tầm thường:

$$\bar{x} = W^\top \bar{y}, \quad \bar{W} = \bar{y} x^\top, \quad \bar{b} = \bar{y},$$

trong đó $\bar{W}$ là một tích ngoài với đúng 16,8 triệu phần tử — bằng kích thước của chính W . Tỉ số 4096 giữa “dựng Jacobian” và “áp dụng Jacobian” là lý do mọi framework *autodiff* đều cài đặt VJP chứ không cài đặt đạo hàm theo nghĩa ma trận. Với một batch X xếp theo hàng, quy tắc trên trở thành $\bar{W} = \bar{Y}^\top X$, và bạn nên coi đây là công thức phải viết lại được từ đầu bất cứ lúc nào.

↪ **trường hợp riêng của A/01** — lan truyền ngược chỉ là quy tắc chuỗi của giải tích nhiều biến, áp dụng theo thứ tự làm chi phí rẻ nhất

↪ **tiền đề C/12/02** — chính tích các Jacobian cục bộ ở đây là đại lượng mà ba hướng chống gradient tiêu biến nhắm vào

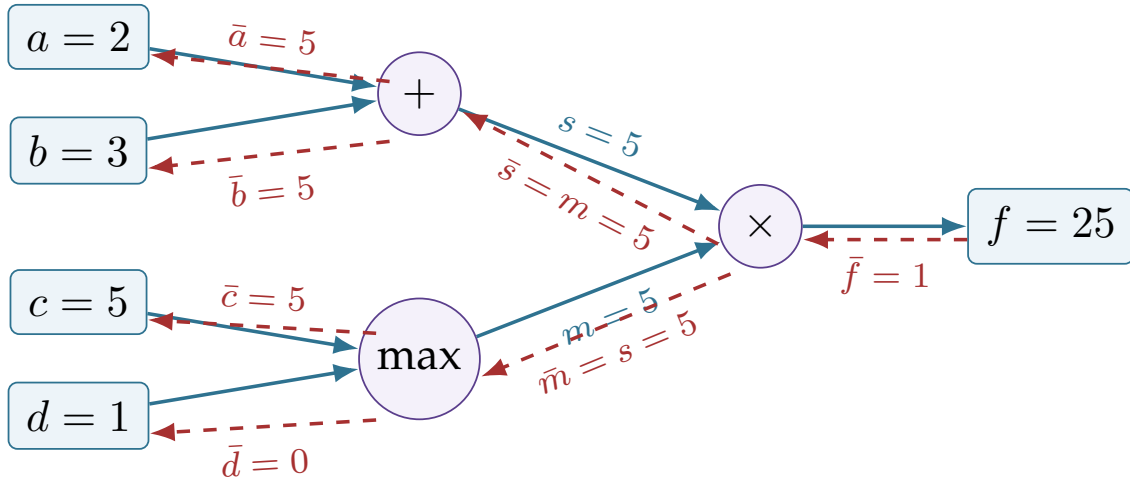
C. Ba mẫu gradient lặp lại (the three gradient patterns)

Nghe qua thì việc phải derive backward cho từng loại tầng là công việc vô tận. Thực tế thì không, vì gần như mọi đồ thị đều được ghép từ ba mẫu, và ba mẫu đó có tên gọi trực quan.

- **Cộng là bộ phân phối** (*add = distributor*) — nếu $y = u + v$ thì $\bar{u} = \bar{v} = \bar{y}$. Nói gọn: adjoint đi qua nút cộng mà không đổi giá trị, chỉ nhân bản ra mọi nhánh.
- **Nhân là bộ hoán đổi** (*multiply = swapper*) — nếu $y = uv$ thì $\bar{u} = \bar{v} \bar{y}$ và $\bar{v} = \bar{u} \bar{y}$. Mỗi nhánh nhận adjoint nhân với giá trị của *nhánh kia*; đây chính là lý do lượt ngược buộc phải nhớ các giá trị của lượt xuôi, và do đó là gốc rễ của chi phí bộ nhớ ở mục G.
- **Max là bộ định tuyến** (*max = router*) — nếu $y = \max(u, v)$ thì toàn bộ adjoint đi về nhánh thắng, nhánh thua nhận 0. Nói gọn: max không trộn, nó chọn đường.
- **Rẽ nhánh là bộ cộng dồn** (*fan-out = accumulator*) — mẫu thứ tư, ít được nêu tên nhưng gây nhiều lỗi nhất. Nếu một giá trị được dùng ở hai chỗ trong đồ thị, adjoint của nó là *tổng* các adjoint quay về từ hai chỗ đó.

Mẫu thứ tư đáng được nói thêm một câu vì hệ quả thực hành của nó rất rộng: *lượt ngược của broadcasting là một phép cộng theo đúng trục đã được broadcast*. Cộng một vector bias shape $(C,)$ vào một *tensor* shape (N, C, H, W) chính là rẽ nhánh giá trị bias ra $N \cdot H \cdot W$ chỗ, nên $\bar{b} = \sum_{n,h,w} \bar{y}_{n,c,h,w}$. Một mình quy tắc này giải thích phần lớn lỗi shape mà người mới gặp khi tự viết lượt ngược.

Hình 1.3 chạy cả ba mẫu trên một ví dụ đủ nhỏ để tính tay. Với $f = (a + b) \cdot \max(c, d)$ và $a = 2, b = 3, c = 5, d = 1$, lượt xuôi cho $s = 5, m = 5, f = 25$. Lượt ngược đặt $\bar{f} = 1$; nút nhân hoán đổi nên $\bar{s} = m = 5$ và $\bar{m} = s = 5$; nút cộng phân phối nên $\bar{a} = \bar{b} = 5$; nút max định tuyến nên $\bar{c} = 5$ và $\bar{d} = 0$. Kiểm tra bằng trực giác: tăng d một chút không đổi gì cả vì d đang thua, đúng như $\bar{d} = 0$; nhưng nếu d vượt c thì đạo hàm nhảy bậc, và đó chính là chỗ kiểm tra gradient bằng số sẽ báo động giả ở phần sau.



Hình 1.3. Ba mẫu gradient trên một đồ thị bốn nút. Mũi tên liền là lượt xuôi, mũi tên đứt là lượt ngược. Nút cộng phân phối adjoint, nút nhân hoán đổi giá trị hai nhánh, nút max định tuyến toàn bộ adjoint về nhánh thắng và cắt nhánh thua về 0.

D. Bốn backward đáng tự viết một lần trong đời

Việc tự derive gradient cho vài tầng cụ thể là bài tập kiểu CS231n, và nó nằm ở tầng bán rã [T2]–[T3]: không phải kiến thức vĩnh cửu, nhưng cũng không phải chi tiết framework sẽ mất giá sau một năm. Giá trị của nó là sau khi làm xong, bạn đọc bất kỳ tầng mới nào cũng tự hỏi “VJP của cái này là gì” thay vì “thư viện có hỗ trợ không”.

Tầng affine đã nêu ở trên. **ReLU** là mẫu max trá hình: $\bar{x} = \bar{y} \cdot \mathbb{I}[x > 0]$, và lượt ngược chỉ cần một mặt nạ bit. **Softmax ghép cross-entropy** là ví dụ đẹp nhất về việc ghép hai phép toán làm backward súp đổ: tính riêng thì Jacobian của softmax là $\text{diag}(p) - pp^\top$, một ma trận $K \times K$ phải nhân với adjoint; ghép lại thì tất cả rút gọn thành $\bar{z} = p - y$, vừa rẻ hơn vừa ổn định hơn về số học. **Max pooling** định tuyến adjoint về vị trí argmax nên phải lưu chỉ số argmax từ lượt xuôi — một chi phí bộ nhớ thật — trong khi average pooling chỉ phân phối $1/k^2$ và không cần nhớ gì.

BatchNorm là bài khó nhất và cũng bổ ích nhất, vì nó là tầng đầu tiên mà gradient của một mẫu phụ thuộc vào *mọi mẫu khác trong batch*. Chuỗi phép toán là trung bình $\rightarrow$ phương sai $\rightarrow$ chuẩn hoá $\rightarrow$ scale-and-shift, và vì μ lẫn σ^2 đều là hàm của cả batch nên adjoint chạy ngược qua ba đường song song rồi cộng lại. Kết quả gọn có dạng

$$\bar{x}_i = \frac{\gamma}{N\sigma} \left(N \bar{\hat{x}}_i - \sum_j \bar{\hat{x}}_j - \hat{x}_i \sum_j \bar{\hat{x}}_j \hat{x}_j \right),$$

với $\hat{x}$ là giá trị đã chuẩn hoá. Hai tổng trong ngoặc chính là hai đường phụ thuộc qua μ và qua σ^2 . Công thức này giải thích luôn một câu hỏi hay bị hỏi: vì sao BatchNorm làm lượt ngược tốn thêm bộ nhớ. Vì để tính được nó, lượt xuôi phải giữ lại $\hat{x}$ và σ cho mỗi tầng BN, chứ không thể vứt đi sau khi dùng.

→ **ràng buộc bởi C/12/04** — việc gradient một mẫu phụ thuộc cả batch là gốc của mọi rắc rối BatchNorm trong huấn luyện phân tán

E. Kiểm tra gradient bằng số (numerical gradient check)

Derive tay thì sai lúc nào không biết, và điều tệ là một gradient sai *vẫn làm loss giảm* — chỉ là giảm chậm hơn hoặc hội tụ về chỗ khác. Nên cần một phép kiểm tra độc lập. Công cụ là **sai phân hữu**

hạn (*finite difference*) đối xứng

$$\frac{\partial f}{\partial x_k} \approx \frac{f(x + he_k) - f(x - he_k)}{2h},$$

chứ không phải sai phân một phía. Lý do là khai triển Taylor: sai phân một phía có sai số bậc $O(h)$, sai phân đối xứng triệt tiêu số hạng bậc hai nên sai số bậc $O(h^2)$. Với $h = 10^{-5}$, đó là chênh lệch giữa sai số cỡ 10^{-5} và cỡ 10^{-10} — đủ để phân biệt “code sai” với “xấp xỉ số”.

Tiêu chí so sánh phải là sai số tương đối $|a - n| / \max(|a|, |n|)$ chứ không phải sai số tuyệt đối, vì độ lớn gradient thay đổi hàng chục bậc giữa các tầng. Kinh nghiệm thực hành phổ biến: dưới 10^{-7} là tốt, quanh 10^{-4} là đáng nghi trừ khi hàm có điểm gãy, trên 10^{-2} gần như chắc chắn sai. Ba điều kiện phải thoả thì con số đó mới có nghĩa. Thứ nhất, phải tính bằng *double*: với float32, epsilon máy khoảng $1,2 \cdot 10^{-7}$, nên phép trừ hai giá trị gần nhau ở tử số mất gần hết chữ số có nghĩa và bạn đang đo nhiễu làm tròn chứ không đo đạo hàm. Thứ hai, phải tắt mọi nguồn ngẫu nhiên — dropout, augmentation, thứ tự batch — hoặc cố định seed, vì f phải là một hàm tắt định thì sai phân mới có nghĩa. Thứ ba, chỉ kiểm vài chục toạ độ ngẫu nhiên, không kiểm hết, vì mỗi toạ độ tốn hai lượt xuôi.

Điểm gãy là cái bẫy lớn nhất. Giả sử x_k rất gần 0 ở đầu vào một ReLU. Khi đó $x_k + h$ và $x_k - h$ nằm hai bên điểm gãy. Sai phân số trả về một giá trị trung gian, còn gradient giải tích trả về 0 hoặc 1. Kiểm tra “thất bại” dù code hoàn toàn đúng. Cách xử lý là dùng ít điểm dữ liệu, chọn toạ độ ở xa điểm gãy, và diễn giải một vài lần thất bại cô lập một cách thận trọng thay vì lao đi viết lại backward. Kỹ năng này không hề mất giá theo thời gian: mỗi lần bạn viết một autograd.Function riêng hay một kernel CUDA cho một phép toán mới, việc đầu tiên nên làm là chạy `torch.autograd.gradcheck` với đầu vào double.

F. Vectorization, và vì sao nó không chỉ là chuyện tốc độ

Bỏ vòng lặp Python khỏi loss và gradient là bài tập kinh điển. Lợi ích hiển nhiên là tốc độ, nhưng chi phí không nằm ở FLOPs. Nó nằm ở phần trên đầu của mỗi lời gọi: thông dịch Python cộng chi phí phóng một kernel, cỡ vài micro-giây. Một nghìn phép toán tí hon vì thế tiêu tốn vài mili-giây trước khi làm phép nhân nào. Đây đúng là hiện tượng bạn đã đo khi tối ưu SLAM: nghẽn ở số lời gọi chứ không ở số phép tính. Nhưng lợi ích thứ hai mới đáng giá lâu dài: viết dạng vector buộc bạn phát biểu rõ shape của mọi thứ, do đó phát biểu rõ *ai rẽ nhánh sang ai*, và khi đó backward gần như tự lộ ra từ ba mẫu ở trên. Đoạn dưới là toàn bộ forward và backward của softmax ghép cross-entropy cho một batch.

```
# Z: (N, K) logits, y: (N,) nhãn đúng
Z = Z - Z.max(axis=1, keepdims=True) # on định so: tranh exp tràn
P = np.exp(Z); P /= P.sum(axis=1, keepdims=True)
loss = -np.log(P[np.arange(N), y]).mean()

dZ = P.copy()
dZ[np.arange(N), y] -= 1.0 # dZ = p - y
dZ /= N # vì loss lay trung bình theo batch
dW = X.T @ dZ # VJP của tang affine
db = dZ.sum(axis=0) # backward của broadcast = cong theo trục
```

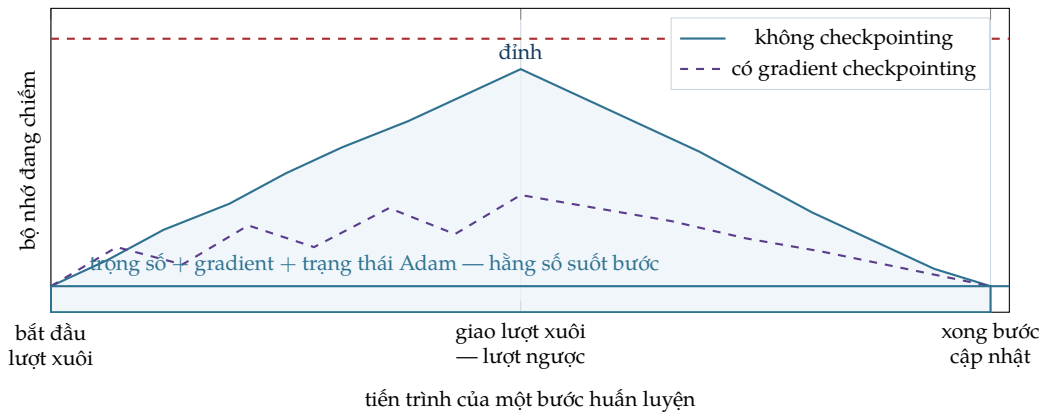
Hai dòng cuối minh hoạ chính xác hai quy tắc đã nêu: tích ngoài cho ma trận trọng số, và tổng theo trục batch cho bias.

G. Cái giá của reverse mode: bộ nhớ kích hoạt

Reverse-mode rẻ về thời gian nhưng không miễn phí, và cái giá của nó là bộ nhớ. Mẫu nhân đòi hỏi giá trị của nhánh kia. Nên lượt ngược cần phần lớn giá trị trung gian của lượt xuôi. Bộ nhớ vì thế tăng theo tổng kích thước **kích hoạt** (*activation*) của cả mạng, không theo số tham số. Một

phép tính cụ thể cho thấy bậc độ lớn. Lấy đầu ra của một tầng conv ở batch 32, 64 kênh, độ phân giải 112×112 , lưu ở fp32. Nó chiếm $32 \cdot 64 \cdot 112 \cdot 112 \cdot 4 \approx 1,03 \cdot 10^8$ byte, tức khoảng 98 MiB cho một tầng. Một backbone vài chục tầng nhanh chóng đưa con số này lên vài GB. Nên trong phần lớn bài huấn luyện thị giác, thứ quyết định batch size tối đa là bộ nhớ chứ không phải tính toán.

→ **ràng buộc bởi** B/11 — bộ nhớ kích hoạt là đại lượng chặn batch size, và chương giới hạn tính toán do nó cùng với băng thông trên mô hình roofline



Hình 1.4. Bộ nhớ trong một bước huấn luyện. Phần không đổi là trọng số, gradient và trạng thái optimizer; phần dâng lên rồi rút xuống là kích hoạt, và nó đạt đỉnh đúng ở chỗ lượt xuôi giao lượt ngược — vì đó là lúc mọi giá trị trung gian đã được sinh ra mà chưa có giá trị nào được giải phóng. Hai hệ quả đọc thẳng từ hình: batch size tối đa bị quyết định bởi một thời điểm chứ không phải bởi mức trung bình, và checkpointing hạ đỉnh bằng cách vớt rồi tính lại, đổi lấy phần diện tích tăng thêm ở lượt ngược. Các con số là minh họa định tính, không phải số đo.

Hình 1.4 giải thích một điều mà con số 98 MiB ở trên không nói: cái chặn bạn không phải tổng bộ nhớ trung bình mà là *đỉnh tức thời* tại điểm giao giữa hai lượt. Đó cũng là lý do một mô hình chạy được ở batch 16 có thể sập ở batch 17 chứ không xuống cấp dần.

Đây là một đánh đổi thời gian-bộ nhớ quen thuộc, và nó có lời giải quen thuộc. **Gradient checkpointing** [2] chỉ lưu một tập con các nút, rồi khi lượt ngược cần một giá trị đã bị vớt thì tính lại nó từ điểm lưu gần nhất; với cách chọn điểm lưu hợp lý, bộ nhớ giảm từ $O(L)$ xuống $O(\sqrt{L})$ với giá khoảng một lượt xuôi phụ, tức chừng 30 phần trăm thời gian. Ở đầu kia của phổ, forward-mode có bộ nhớ hằng số nhưng cần một lượt cho mỗi hướng đầu vào, nên nó không dùng để huấn luyện mà dùng cho tích Jacobian-vector, chẳng hạn khi cần tích Hessian-vector trong các phương pháp bậc hai.

H. Giới hạn và trường hợp hỏng

Khung đồ thị có ba chỗ gãy đáng biết trước.

Thứ nhất là *điểm không khả vi*. ReLU tại 0 và max khi hai nhánh bằng nhau không có đạo hàm; framework chọn một quy ước, thường là 0 cho ReLU và “nhánh chỉ số nhỏ hơn thắng” cho max. Quy ước đó vô hại về mặt tối ưu hoá nhưng có hệ quả về tính tái lập: khi hai giá trị bằng nhau tới từng bit, việc ai thắng phụ thuộc thứ tự duyệt, và thứ tự duyệt có thể phụ thuộc lịch trình luồng. Đây đúng là loại nguồn bất định mà bạn đã truy trong hệ SLAM đa luồng, chỉ là xuất hiện ở một tầng khác.

Thứ hai là *đồ thị bị cắt mà không ai báo*. Một phép gán tại chỗ ghi đè giá trị mà lượt ngược cần sẽ cho gradient sai; PyTorch bắt được phần lớn trường hợp này bằng bộ đếm phiên bản tensor, nhưng một kernel tự viết thì không có ai bảo vệ và bạn nhận gradient sai một cách im lặng. Cùng loại đó là việc vô tình gọi `detach()` hoặc đặt một nhánh trong `no_grad()`: loss vẫn giảm, chỉ có một mô-đun không bao giờ học, và triệu chứng duy nhất là một đường cong hơi tệ hơn kỳ vọng.

Thứ ba là *ổn định số*. $\log(0)$, $\exp$ tràn, chia cho phương sai gần 0: ba nguồn NaN kinh điển. Chúng không phải lỗi của autodiff mà là lỗi của biểu thức bạn viết ra. Cách chữa luôn là viết lại biểu thức ở dạng ổn định: trừ max trước khi lấy $\exp$, dùng $\log\text{sumexp}$, cộng ε vào mẫu số. Kẹp gradient sau khi nó đã thành NaN thì quá muộn. Cùng họ với nó là điều khiển luồng phụ thuộc dữ liệu: một câu lệnh rẽ nhánh theo giá trị tensor là hợp lệ, nhưng gradient chỉ chảy qua nhánh đã chọn, nên nếu bạn tưởng mình đang huấn luyện cả hai nhánh thì bạn nhầm.

CẠM BÃY

Cái bẫy sinh ra nhiều gradient sai âm thầm nhất không phải công thức khó, mà là quên rằng *broadcast ở lượt xuôi là rẽ nhánh, nên lượt ngược phải cộng theo trục đã broadcast*. Nguy hiểm ở chỗ nếu bạn quên phép cộng đó, shape của kết quả nhiều khi vẫn hợp lệ nhờ chính broadcast, nên không có exception nào được ném ra; bạn chỉ nhận một gradient sai hệ số N và một quá trình huấn luyện chậm hơn đáng kể. Cách phòng duy nhất đáng tin là kiểm tra bằng số trên một ví dụ có $N = 3$ và $N = 1$ rồi so hai kết quả.

CẦN NHỚ

Lan truyền ngược = duyệt đồ thị theo thứ tự tô-pô đảo, mỗi nút áp một tích vector-Jacobian cục bộ, với ba mẫu cộng-nhân-max cộng thêm quy tắc “rẽ nhánh thì cộng dồn”. Nếu bạn gọi tên được Jacobian cục bộ của một khối, bạn vi phân được khối đó — kể cả khối bạn chưa từng thấy. [T1]

I. Kết nối

Mục này cho bạn công cụ tính gradient; mục tiếp theo, [C/12/02-lam-cho-training-chay-duoc](#), hỏi câu kè: gradient tính đúng rồi thì làm sao để nó không chết hoặc nổ khi đi qua năm mươi tầng — và câu trả lời bắt đầu chính từ tích các Jacobian cục bộ vừa nêu ở đây. Chi phí bộ nhớ kích hoạt ở cuối mục là tiền đề trực tiếp cho [B/11-gioi-han-tinh-toan](#) (roofline, chế độ nghẽn bộ nhớ) và cho phần huấn luyện phân tán ở [C/12/04-tong-quat-hoa-va-chan-doan](#). Ý “mọi kiến trúc chỉ là một đồ thị” được dùng lại nguyên vẹn ở [C/13-kien-truc-backbone](#): khi đọc ResNet hay Transformer, thứ đáng hỏi không phải sơ đồ khối mà là đường đi của adjoint qua sơ đồ đó.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao reverse-mode rẻ hơn forward-mode đúng bằng tỉ số giữa số đầu vào và số đầu ra, và trong tình huống nào tỉ số đó đảo chiều khiến forward-mode thắng?
2. Vì sao không bao giờ dựng ma trận Jacobian của một tầng affine, dù về mặt toán học nó tồn tại?
3. Kiểm tra gradient bằng số báo sai số tương đối $3 \cdot 10^{-3}$ trên một mạng có ReLU — hai nguyên nhân khả dĩ là gì, và bạn phân biệt chúng bằng cách nào?
4. Vì sao gradient của BatchNorm với một mẫu lại phụ thuộc vào các mẫu khác trong batch, và điều đó kéo theo hệ quả gì cho huấn luyện phân tán?
5. Bạn viết một kernel CUDA cho một phép toán mới và lượt ngược cho gradient sai đúng hệ số bằng batch size. Chỗ nào trong code là nghi phạm số một?
6. Gradient checkpointing đổi cái gì lấy cái gì, và khi nào phép đổi đó *không* đáng?

Nguồn

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.

1.3 Làm cho training chạy được (Making Training Work)

PHỤ THUỘC C/12/01-co-hoc-cua-gradient (tích các Jacobian cục bộ — toàn bộ mục này là hệ quả của nó); A/01-toan-toi-thieu (phương sai, giá trị kỳ dị, số điều kiện).

TÓM TẮT

Mục này trả lời câu hỏi trung tâm của cả chương: làm sao đưa *gradient* qua năm mươi tầng mà nó không chết cũng không nổ. Ba nhóm kỹ thuật tưởng như rời rạc — hàm kích hoạt, khởi tạo trọng số, normalization — thực ra nhắm vào đúng một đại lượng duy nhất. Đọc xong bạn nhìn một file cấu hình huấn luyện và nói được từng dòng trong đó đang giữ đại lượng nào ổn định, thay vì chép nguyên recipe của người khác. Nếu chỉ nhớ một điều: adjoint đi ngược qua L tầng là tích của L Jacobian, nên mọi thứ trong mục này chỉ là các cách khác nhau để ép hệ số nhân của tích đó về gần 1.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tầng (layer)** — một khối phép toán có tham số riêng; “mạng sâu” nghĩa là nhiều tầng nối tiếp, và toàn bộ khó khăn của mục này sinh ra từ chữ “nối tiếp” đó.
- **kích hoạt (activation)** — giá trị đầu ra của một tầng giữa mạng. “Hàm kích hoạt” là phép phi tuyến tạo ra nó, ví dụ ReLU — hai khái niệm khác nhau nhưng tên gần giống.
- **gradient** — vector đạo hàm riêng của hàm mất mát theo từng tham số; đi ngược nó thì mất mát giảm. Gradient “tiêu biến” nghĩa là nó nhỏ tới mức tầng đó không học được nữa.
- **phương sai (variance)** — đại lượng đo độ tản mát của một tập số quanh trung bình của chúng; cả lập luận khởi tạo trọng số trong mục này chỉ là theo dõi phương sai đi qua từng tầng.
- **số điều kiện (condition number)** — tỉ số giữa độ cong lớn nhất và nhỏ nhất của mặt mất mát; càng lớn thì đường đồng mức càng dẹt và thuật toán tối ưu càng đi zigzag.
- **batch (lô)** — nhóm mẫu đi qua mạng cùng lúc. Với BatchNorm, kích thước lô không chỉ là chuyển tốc độ mà còn thay đổi *chính hàm số* mà mạng biểu diễn.
- **chế độ huấn luyện và suy luận (train / eval mode)** — cùng một mô hình hành xử khác nhau ở hai chế độ: BatchNorm và dropout dùng thống kê batch khi huấn luyện, dùng thống kê tích lũy khi suy luận. Quên chuyển chế độ là bug kinh điển ở cuối mục.
- **bias — phân biệt hai nghĩa** — trong $Wx + b$, b là tham số cộng của một tầng, và “đẩy bias xuống rất âm” ở phần ReLU là nói về nó. Nghĩa thống kê — độ lệch hệ thống của một bộ ước lượng — không dùng trong mục này.

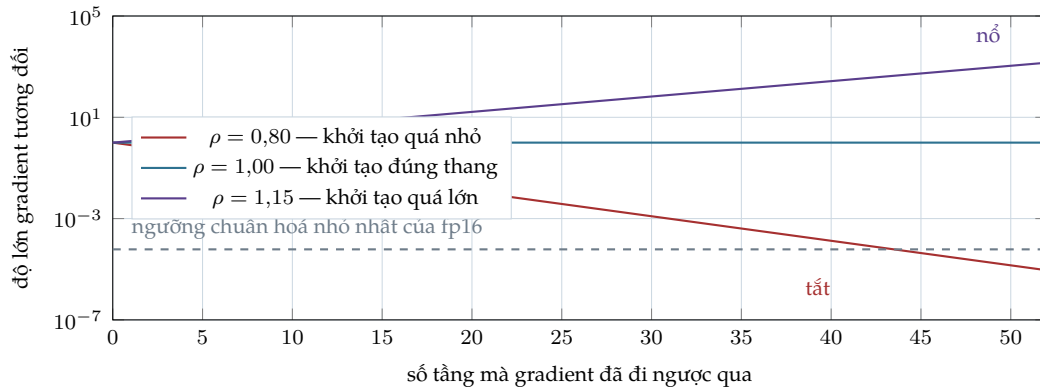
A. Đặt vấn đề

Gradient đã tính đúng, kiểm tra bằng số đã qua, code chạy không lỗi. Vậy mà loss vẫn nằm im ở mức phỏng đoán ngẫu nhiên, hoặc bay lên NaN ở bước thứ ba trăm. Đây là trạng thái mà mọi người từng huấn luyện mạng sâu đều đã sống qua, và nó không phải một lỗi lập trình. Nó là biểu hiện của nút thắt lịch sử lớn nhất của ngành: *vanishing and exploding gradient* — gradient tắt dần hoặc nổ tung khi đi ngược qua nhiều tầng. Giữa đầu thập niên 1990 và khoảng 2012, người ta biết cách viết mạng sâu và biết cách tính gradient của nó. Chỉ có điều không ai huấn luyện được nó. Gần như mọi “trick” trong một file cấu hình huấn luyện hiện đại tồn tại vì lý do đó.

Cơ chế thì rất gọn, và nó suy trực tiếp ra từ mục trước. Adjoint đi ngược qua L tầng là một tích của L Jacobian cục bộ. Nếu giá trị kỳ dị điển hình của mỗi Jacobian là ρ , thì độ lớn gradient ở

tầng đầu tiên tỉ lệ với ρ^L . Đây là hàm mũ, nên không có vùng an toàn rộng: với $L = 50$, $\rho = 0,9$ cho $0,9^{50} \approx 5 \cdot 10^{-3}$ và $\rho = 1,1$ cho $1,1^{50} \approx 117$. Một sai lệch 10 phần trăm ở mỗi tầng biến thành hai bậc độ lớn ở toàn mạng theo hướng này và hai bậc theo hướng kia.

↪ hệ quả của C/12/01 — tích các Jacobian là kết quả trực tiếp của việc adjoint đi ngược qua từng tầng một



Hình 1.5. Vì sao không có vùng an toàn rộng cho khởi tạo. Ba đường chỉ khác nhau ở hệ số ρ của một tầng — lệch 20 phần trăm theo hai chiều — nhưng sau 50 tầng chúng cách nhau tám bậc độ lớn, vì lũy thừa thắng mọi thứ khác. Đường $\rho = 0,80$ chui xuống dưới ngưỡng biểu diễn của fp16 từ khoảng tầng thứ 40, tức các tầng đầu mạng không nhận được tín hiệu nào cả. Đây là đồ thị của một mô hình đơn giản hoá (mọi Jacobian có cùng phổ, độc lập nhau) nên nó cho thấy *chiều* và *tốc độ* của hiện tượng, không dự đoán giá trị cụ thể.

Hình 1.5 là toàn bộ lý do ba mục tiếp theo tồn tại: khoảng cách giữa “huấn luyện được” và “không huấn luyện được” chỉ là vài phần trăm ở hệ số của một tầng.

Do đó bài toán không phải “làm cho ρ tốt” mà là “làm cho ρ bằng 1 và giữ nó ở đó suốt quá trình huấn luyện”.

Có đúng ba hướng tấn công vào cùng con số ρ đó, và toàn bộ mục này là ba hướng ấy.

- **Sửa hệ số danh nghĩa** — chọn hàm kích hoạt có đạo hàm không nhỏ đi (mục C).
- **Đặt điểm làm việc ban đầu** — chọn khởi tạo sao cho phương sai tín hiệu giữ nguyên qua mỗi tầng (mục D).
- **Hồi tiếp trong lúc chạy** — chèn phép chuẩn hoá ở giữa mạng để ép lại phân bố mỗi khi nó trôi (mục E).

Nhưng trước cả ba, có một việc rẻ hơn nhiều phải làm trước ở đầu vào. Giá trị cụ thể của cả mục: bạn sẽ đọc được một cấu hình huấn luyện lạ và chỉ ra chính xác dòng nào đang giữ ρ gần 1, do đó biết dòng nào có thể bỏ và dòng nào không.

TRỰC GIÁC

Coi mạng sâu như một chuỗi năm mươi bộ khuếch đại nối tiếp nhau, mỗi bộ có hệ số khuếch đại danh nghĩa là 1 nhưng lệch chuẩn vài phần trăm. Không ai chỉnh từng bộ để tổng thể đúng bằng 1 — sai số nhân dồn sẽ thắng. Cái người ta làm trong điện tử là mắc hồi tiếp âm quanh mỗi tầng để hệ số của tầng đó bị ép về giá trị mong muốn bất kể linh kiện trôi. Ba hướng trong mục này là ba kiểu hồi tiếp khác nhau lên cùng một đại lượng: hàm kích hoạt sửa hệ số danh nghĩa, khởi tạo đặt điểm làm việc ban đầu, normalization là vòng hồi tiếp thật sự vì nó đo phân bố đang chạy rồi kéo nó về.

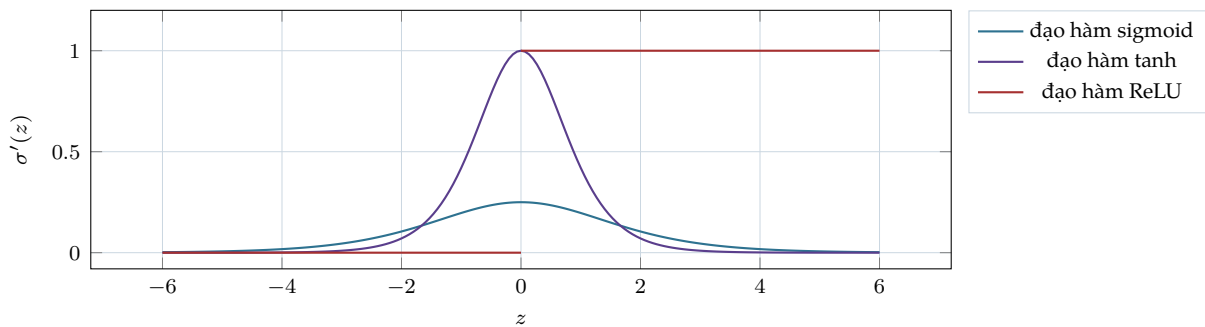
B. Chuẩn hoá đầu vào (input normalization): việc rẻ nhất, làm trước tiên

Trước khi nói về chiều sâu, hãy nhìn một tầng tuyến tính duy nhất. Hessian của mất mát bình phương theo trọng số của tầng đó tỉ lệ với $\mathbb{E}[xx^T]$, tức ma trận moment bậc hai của đầu vào. Giả sử một đặc trưng nằm trong $[0, 1]$ còn đặc trưng kia trong $[0, 1000]$. Hai trị riêng của Hessian khi đó lệch nhau khoảng 10^6 , và đường đồng mức của mất mát là những ellipse cực dẹt. Gradient descent trên một ellipse dẹt đi zigzag. Bước học phải đủ nhỏ để không phân kỳ theo trục dọc, nên nó nhỏ đến vô lý theo trục hoành. Tốc độ hội tụ khi đó do **số điều kiện** (condition number) κ chi phối, theo hệ số $\left(\frac{\kappa-1}{\kappa+1}\right)^t$. Với $\kappa = 1000$, hệ số đó là 0,998 mỗi bước — cần khoảng 350 bước chỉ để giảm sai số đi một nửa.

Trừ trung bình và chia độ lệch chuẩn theo từng kênh đưa κ từ 10^6 về vài đơn vị với chi phí gần bằng không. Đây là lý do vì sao nó là dòng đầu tiên của mọi pipeline, chứ không phải vì một lý do thẩm mỹ nào. Bước xa hơn về lý thuyết là *whitening* bằng PCA, đưa κ về đúng 1; nhưng với ảnh thì nó gần như không bao giờ được dùng, vì hai lý do có thể tính ra được. Thứ nhất là chi phí. Một ảnh $3 \times 224 \times 224$ có 150 528 chiều, nên ma trận hiệp phương sai của nó có $150\,528^2 \approx 2,27 \cdot 10^{10}$ phần tử. Riêng việc lưu nó đã tốn khoảng 90 GB ở fp32, chưa nói tới phân rã. Thứ hai, và quan trọng hơn, whitening trộn mọi pixel với mọi pixel nên nó *phá cấu trúc không gian*: sau khi whiten, hai pixel cạnh nhau không còn cạnh nhau theo nghĩa thống kê, và toàn bộ giả định **chia sẻ trọng số** (weight sharing) của *convolution* mất cơ sở. Đây là một ví dụ sạch về nguyên lý “một phép biến đổi tối ưu theo một tiêu chí có thể phá huỷ đúng cấu trúc mà phần còn lại của hệ thống đang dựa vào”.

C. Hướng 1 — hàm kích hoạt (kích hoạt function), đọc như một chuỗi sửa lỗi

Hàm kích hoạt (activation function) đóng góp trực tiếp vào ρ : Jacobian của nó là một ma trận đường chéo chứa $\sigma'(z)$, nên nếu σ' nhỏ thì ρ nhỏ. Sigmoid có σ' cực đại bằng 0,25 tại $z = 0$ và tiến nhanh về 0 ở hai đầu. Ngay cả ở điều kiện tốt nhất, mười tầng sigmoid nhân vào nhau cho hệ số $0,25^{10} \approx 10^{-6}$; ở điều kiện thực tế, nơi phần lớn nơ-ron bão hoà, con số còn tệ hơn nhiều. Một phép nhân hai dòng như vậy giải thích vì sao mạng sâu không huấn luyện được trước 2010, thuyết phục hơn bất kỳ lời giải thích định tính nào.



Hình 1.6. Đạo hàm của ba hàm kích hoạt — đây là đại lượng thật sự đi vào tích các Jacobian, chứ không phải bản thân hàm. Đỉnh của đạo hàm sigmoid chỉ là 0,25 và tụt nhanh về 0 ở hai đầu; đạo hàm tanh đạt 1 nhưng cũng bão hoà; ReLU giữ đúng 1 trên toàn nửa dương và trả về 0 trên nửa âm, đổi tính bão hoà lấy nguy cơ nơ-ron chết.

Hình 1.6 vẽ đúng đại lượng cần nhìn: không phải hình dạng hàm kích hoạt mà là hình dạng *đạo hàm* của nó, vì chỉ đạo hàm mới đi vào tích Jacobian. Bảng 1.3 đọc tiếp bức tranh đó theo trục thời gian.

Bảng 1.3. Chuỗi hàm kích hoạt đọc như một chuỗi sửa lỗi. Cột cuối là cột đáng nhớ: mỗi bước sửa được một khiếm khuyết và để lại một khiếm khuyết mới.

Hàm	Cơ chế	Sửa được gì	Vấn hòng ở đâu
Sigmoid	Ép về $(0, 1)$	— (khởi điểm lịch sử)	Bão hoà hai đầu giết gradient; đầu ra không quanh 0 nên gradient của cả tầng cùng dấu
Tanh	Ép về $(-1, 1)$	Đầu ra quanh 0, hết hiện tượng cùng dấu	Vẫn bão hoà, $\sigma' \leq 1$
ReLU	$\max(0, x)$	Không bão hoà phía dương ($\sigma' = 1$), tính cực rẻ	Nơ-ron chết vĩnh viễn nếu rơi hẳn sang phía âm; không quanh 0
Leaky ReLU, PReLU, ELU	Cho phía âm một độ dốc nhỏ	Nơ-ron chết	Thêm siêu tham số; lợi ích không ổn định giữa các bài toán
GELU, SiLU	Cổng mượt theo giá trị đầu vào	Đạo hàm liên tục, hợp với transformer và lịch học dài	Đắt hơn ReLU vài chục phần trăm; khác biệt nhỏ trên CNN

Bảng 1.3 đọc theo chiều dọc thì thấy rõ đây không phải một danh sách lựa chọn ngang hàng mà là một chuỗi nhân quả. Điều đáng chú ý là bước tiến lớn nhất — từ tanh sang ReLU — không đến từ một hàm “tốt hơn” theo nghĩa toán học mà từ việc *bỏ đi* tính bão hoà. Đáng chú ý không kém là ReLU tạo ra một chế độ hỏng hoàn toàn mới không tồn tại trước đó: một nơ-ron nhận bước cập nhật lớn đẩy bias xuống rất âm sẽ cho 0 trên mọi dữ liệu, do đó gradient của nó bằng 0 mãi mãi, và nó chết không hồi phục. Tỷ lệ nơ-ron chết (*dead ReLU*) vì thế là một chỉ báo sức khoẻ đáng đo, và ta sẽ gặp lại nó ở mục sau.

D. Hướng 2 — khởi tạo trọng số (weight initialization), hay toán đơn giản giải bài toán thật

Lập luận phương sai là ví dụ đẹp nhất trong ngành về việc vài dòng đại số giải quyết một vấn đề đã chặn cả một lĩnh vực. Xét $y = Wx$ với n đầu vào, các w_{ij} độc lập cùng phân bố trung bình 0, và x độc lập với W . Khi đó

$$\text{Var}(y_j) = n \text{Var}(w) \text{Var}(x).$$

Muốn **phương sai** (*variance*) tín hiệu không đổi qua mỗi tầng, cần $n \text{Var}(w) = 1$, tức $\text{Var}(w) = 1/n_{\text{in}}$. Nhưng lượt ngược cũng phải giữ phương sai, và ở đó vai trò của n_{in} và n_{out} đổi chỗ; không thoả cả hai cùng lúc được nên Xavier/Glorot chọn trung hoà $\text{Var}(w) = 2/(n_{\text{in}} + n_{\text{out}})$ [3]. Đó là lời giải cho mạng tanh.

Với ReLU, lập luận trên thiếu một hệ số. ReLU xoá một nửa số đầu ra về 0, nên phương sai sau kích hoạt chỉ còn một nửa phương sai trước nó; hệ số mỗi tầng là $1/2$ chứ không phải 1. Sau 30 tầng, tín hiệu bị nhân với $2^{-30} \approx 10^{-9}$ — tức là tắt hẳn, dù bạn đã “khởi tạo đúng” theo Xavier. Sửa lại thì hiển nhiên: nhân đôi phương sai, $\text{Var}(w) = 2/n_{\text{in}}$. Đó là khởi tạo He/Kaiming [4], và chính hệ số 2 đó là thứ cho phép huấn luyện mạng ReLU ba mươi tầng lần đầu tiên. Hãy để ý mạch lập luận: một giả định (“ReLU không đổi phương sai”) bị vi phạm, hậu quả được lượng hoá ra một con số, và lời sửa là một hằng số. Đây là dạng suy luận mà bạn nên tái tạo được, chứ không phải hai công thức để nhớ.

↔ **tiền đề** A/01 — toàn bộ lập luận chỉ dùng một tính chất của phương sai: phương sai của tổng các biến độc lập bằng tổng phương sai

E. Hướng 3 – normalization, vòng hồi tiếp thật sự

Hai hướng trên chỉ đặt điều kiện *ban đầu*. Sau vài nghìn bước, trọng số đã đổi và phân bố kích hoạt trôi khỏi chỗ được thiết kế. **Chuẩn hoá** (*normalization*) đo phân bố đang chạy rồi ép nó về trung bình 0 và độ lệch chuẩn 1. Sau đó nó trả lại quyền chính cho mạng, qua hai tham số học được γ và β . Bốn biến thể khác nhau ở đúng một điểm: *lấy trung bình trên trục nào*.

Bảng 1.4. Bốn biến thể normalization. Chúng chỉ khác nhau ở tập chỉ số dùng để tính thống kê, và chính lựa chọn đó quyết định chúng hỏng khi nào.

Loại	Trung bình theo	Sinh ra để giải quyết	Hỏng khi nào
BatchNorm [5]	Batch $\times$ không gian, riêng từng kênh	Ổn định phân bố giữa mạng, cho phép dùng learning rate lớn hơn hẳn	Batch nhỏ (thống kê nhiễu); lệch train/eval; phải đồng bộ giữa các GPU
LayerNorm [6]	Toàn bộ đặc trưng của <i>một</i> mẫu	Batch nhỏ hoặc bằng 1; chuỗi độ dài thay đổi; không phụ thuộc mẫu khác	Trên CNN thì giác thường kém BN; trộn chung các kênh có ý nghĩa rất khác nhau
GroupNorm [7]	Nhóm kênh của một mẫu	Detection/segmentation với batch một tới hai ảnh mỗi GPU	Thêm siêu tham số số nhóm; không có lợi thế nào khi batch đủ lớn
InstanceNorm	Mỗi kênh của mỗi mẫu	Style transfer: xoá thống kê style khỏi đặc trưng	Ở bài toán khác, thứ bị xoá lại là thông tin có ích (độ sáng, tương phản)

Bảng 1.4 nên đọc cùng một nhận xét về mặt cơ chế: câu chuyện gốc của BatchNorm là “giảm internal covariate shift”, nhưng lời giải thích đó đã bị chất vấn nghiêm túc. Santurkar và cộng sự bơm nhiễu để *tăng* covariate shift một cách cố ý, và BN vẫn hoạt động tốt. Họ đề xuất tác dụng thật là làm mất mát mát hơn. Gradient bớt thay đổi đột ngột, nên bước đi dài hơn mà không phân kỳ [8]. Đây là một trường hợp đáng nhớ khi đọc paper. Kỹ thuật đúng, nhưng lời giải thích đi kèm nó lúc ra đời có thể sai. Học thuộc lời giải thích thay vì cơ chế thì bạn sẽ suy luận sai ở tình huống kế tiếp.

Hệ quả kỹ thuật quan trọng nhất của BatchNorm là nó *ghép các mẫu trong batch lại với nhau*. Ta đã thấy điều đó ở công thức backward trong mục trước: gradient của một mẫu phụ thuộc mọi mẫu cùng batch. Từ đó suy ra gần như mọi chế độ hỏng của nó. Batch nhỏ thì thống kê nhiễu, nên gradient nhiễu. Lúc suy luận không có batch nên phải dùng thống kê chạy tích lũy trong huấn luyện, và bất kỳ chênh lệch nào giữa hai chế độ đều thành sai lệch hệ thống. Trong huấn luyện phân tán, mỗi tiến trình tính thống kê riêng nên mô hình thực chất khác nhau ở mỗi GPU trừ khi bạn dùng SyncBN. LayerNorm và GroupNorm né toàn bộ họ vấn đề này bằng cách không dùng tới trục batch — và đó là toàn bộ lý do transformer dùng LayerNorm, chứ không phải vì LayerNorm “hiện đại hơn”.

→ **ràng buộc bởi** C/14 — detection chạy một tới hai ảnh mỗi GPU, nên chính ràng buộc batch nhỏ ở đó quyết định lựa chọn GroupNorm

(1) conv → chuẩn hoá → hàm kích hoạt — thứ tự mặc định



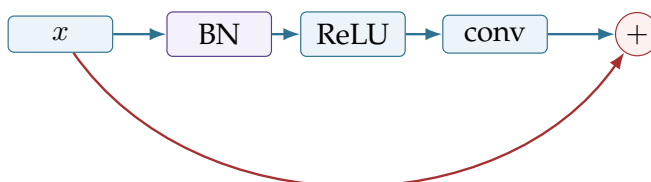
Đầu vào ReLU có trung bình 0, nên khoảng một nửa số nơ-ron hoạt động ở mọi batch. Đây là cấu hình mà lập luận phương sai của He giả định.

(2) conv → hàm kích hoạt → chuẩn hoá



BN phải chuẩn hoá một phân bố *không âm*, nên β phải gánh việc dời tâm và γ làm việc trên một phổ hẹp hơn. Chạy được, nhưng lãng phí hai tham số.

(3) tiền kích hoạt trong khối residual: BN → ReLU → conv



Gradient từ đầu ra về x đi qua một đường không bị chuẩn hoá chặn, nên thành phần bằng 1 trong Jacobian được giữ nguyên qua rất nhiều khối.

đường tắt *sạch*: không có phép toán nào chen vào

Hình 1.7. Ba cách đặt phép chuẩn hoá quanh hàm kích hoạt, và hệ quả lên luồng gradient. Điều hình này nói mà câu văn khó nói: vị trí của khối BN không phải chi tiết thẩm mỹ. Nó quyết định phân bố mà ReLU nhìn thấy, tức hàng 1 so với hàng 2. Nó cũng quyết định đường tắt có sạch hay không, tức hàng 3. Ở hàng 3, mọi phép toán đã bị đẩy vào *nhánh residual*, nên đường cộng từ x tới điểm $+$ hoàn toàn trống. Đó là lý do biến thể tiền kích hoạt huấn luyện được ở độ sâu lớn hơn hẳn.

Hình 1.7 cũng là chỗ ba hướng của mục này gặp nhau trên một sơ đồ duy nhất: hàng 1 nói về quan hệ giữa chuẩn hoá và hàm kích hoạt, hàng 3 nói về quan hệ giữa chuẩn hoá và kết nối tắt.

Còn một hướng thứ tư mà mục này cố tình không kể: **kết nối tắt** (*residual connection*). Nó tấn công đúng cùng đại lượng ρ , nhưng bằng cách thêm một đường cộng để Jacobian có dạng $I + \partial F / \partial x$ — luôn có một thành phần bằng 1 không thể triệt tiêu. Vì nó là một lựa chọn *kiến trúc* chứ không phải một thủ thuật huấn luyện, nó thuộc về C/13/01-dong-cnn; nhưng khi đọc chương đó, hãy nhớ nó là thành viên thứ tư của đúng gia đình này.

↔ cùng nguyên lý với C/13/01 — kết nối tắt giữ tích Jacobian gần 1 bằng một đường cộng, tức cùng mục tiêu với ba hướng ở đây nhưng thực hiện ở tầng kiến trúc

F. Giới hạn và trường hợp hỏng

Ba hướng trên không cộng dồn vô hạn, và chúng có những chỗ giao thoa gây bất ngờ.

Khởi tạo đúng chỉ đúng ở bước 0. He/Kaiming giả định trọng số độc lập cùng phân bố và đầu vào đã chuẩn hoá; sau vài nghìn bước cả hai giả định đều sai, nên khởi tạo không thay được normalization. Ngược lại, ở các mạng rất sâu có kết nối tắt, khởi tạo quan trọng theo một kiểu khác. Nhánh residual cần được khởi tạo nhỏ hoặc bằng 0, để mạng bắt đầu gần với ánh xạ đồng nhất. Bỏ chi tiết đó đi thì mạng sâu vẫn khó huấn luyện, dù đã có đủ BN.

Normalization che giấu lỗi. Một mạng có BN ở khắp nơi vẫn huấn luyện được dù khởi tạo sai, dù đầu vào chưa chuẩn hoá, dù learning rate lệch một bậc — chỉ là kém hơn đáng kể. Đây là tin tốt cho tiến độ và tin xấu cho việc học chẩn đoán, vì nó xoá mất các triệu chứng mà lẽ ra bạn phải học đọc.

Cuối cùng, các con số như ρ^L hay 2^{-30} trong mục này là ước lượng bậc độ lớn dựa trên giả định các Jacobian độc lập và có phổ đều. Giả định đó thô: trong thực tế các Jacobian tương quan với nhau, và phổ của chúng không đều. Chúng dùng để cho thấy *hàm mũ thắng mọi thứ khác*, chứ không dùng để dự đoán một giá trị cụ thể.

CẠM BÃY

Bug kinh điển của cả nhóm này: quên gọi `model.eval()` khi đánh giá. BatchNorm khi đó dùng thống kê của chính batch kiểm thử thay vì thống kê chạy. Kết quả vì thế phụ thuộc vào việc bạn xếp mẫu nào chung batch. Tệ nhất là nó thường *tốt hơn* thực tế, vì mô hình được nhìn trộm thống kê của tập kiểm thử. Triệu chứng nhận dạng: điểm số đôi khi bạn đổi batch size lúc đánh giá, hoặc điểm số tụt hẳn khi triển khai với batch bằng 1. Cùng một dòng lệnh cũng tắt dropout, nên hai lỗi này luôn đi cùng nhau.

CẦN NHỚ

Cả ba hướng — hàm kích hoạt, khởi tạo, normalization — chỉ nhắm vào đúng một đại lượng: hệ số nhân của tích các Jacobian qua các tầng. Mọi câu hỏi dạng “kỹ thuật X có cần thiết không” nên được dịch lại thành “trong cấu hình của tôi, cái gì đang giữ hệ số đó gần 1”. [T1]

G. Kết nối

Mục này lấy đầu vào từ [C/12/01-co-hoc-cua-gradient](#): toàn bộ lập luận chỉ là hệ quả số học của việc adjoint là một tích Jacobian. Nó chuyển tiếp sang [C/12/03-toi-uu-hoa](#). Ở đó ta giả định gradient đã lành mạnh, rồi hỏi tiếp nên đi bước dài bao nhiêu. Warmup sẽ xuất hiện ngay tại đó, như hệ quả trực tiếp của việc thống kê BN chưa ổn định ở những bước đầu. Thành viên thứ tư của gia đình, kết nối tắt, nằm ở [C/13/01-dong-cnn](#). Lựa chọn giữa BatchNorm và GroupNorm gắn chặt với ràng buộc batch size trong detection, nên nó được nhắc lại ở [C/14-bai-toan-cv-loi](#); còn chi phí bộ nhớ của việc lưu $\hat{x}$ cho lượt ngược thuộc về [B/11-gioi-han-tinh-toan](#).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một sai lệch 10 phần trăm ở hệ số mỗi tầng lại là vấn đề nghiêm trọng, trong khi sai lệch 10 phần trăm ở learning rate thì không?
2. Whitening đưa số điều kiện về 1, tức tối ưu theo tiêu chí ở đầu mục. Vậy vì sao gần như không ai whiten ảnh?
3. Hệ số 2 trong khởi tạo He đến từ đâu, và nếu bạn thay ReLU bằng GELU thì lập luận đó thay đổi thế nào?
4. BatchNorm hỏng theo ba kiểu khác nhau; cả ba đều suy ra từ một tính chất duy nhất của nó. Tính chất đó là gì?
5. Vì sao lời giải thích “internal covariate shift” bị nghi ngờ, và vì sao việc học thuộc một lời giải thích sai lại nguy hiểm hơn là không có lời giải thích nào?
6. Bạn thay BatchNorm bằng LayerNorm trong một CNN thị giác và kết quả tệ đi — cách giải thích của bạn là gì, và phép đo nào kiểm chứng được nó?

Nguồn

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.

1.4 Tối ưu hoá và siêu tham số (Optimization and Hyperparameters)

PHỤ THUỘC C/12/02-lam-cho-training-chay-duoc (gradient phải lành mạnh trước khi bàn tới bước đi); A/01-toan-toi-thieu (số điều kiện, hội tụ của gradient descent).

TÓM TẮT

Mục này đọc bảy *optimizer* quen thuộc như một chuỗi vá lỗi liên tiếp chứ không như một menu lựa chọn, rồi chỉ ra siêu tham số nào thật sự quyết định kết quả. Đọc xong bạn nhìn một đường cong học và đoán được đang sai ở đâu, đặt được *learning rate* và lịch của nó có căn cứ thay vì mò, và giải thích được vì sao *mixed precision* cần *loss scaling*. Nếu chỉ nhớ một điều: optimizer chỉ định hình lại bước đi, nó không cứu được một learning rate sai — nên hãy tune learning rate trước, batch size sau, và chọn optimizer cuối cùng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán; huấn luyện là đi tìm bộ tham số làm dự đoán sát nhất.
- **gradient** — vectơ đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm mất mát TĂNG nhanh nhất, nên mọi thuật toán ở đây đi ngược lại nó.
- **learning rate** — độ dài bước mà thuật toán tối ưu đi theo hướng ngược gradient ở mỗi lần cập nhật; quá lớn thì nghiệm bật ra ngoài, quá nhỏ thì hội tụ chậm tới mức vô dụng.
- **optimizer (thuật toán tối ưu)** — quy tắc biến gradient thành bước cập nhật thực tế; toàn bộ mục này là lịch sử bảy quy tắc như vậy và lý do từng cái ra đời.
- **siêu tham số (hyperparameter)** — con số do người đặt trước khi huấn luyện chứ không được học từ dữ liệu — learning rate, batch size, cường độ weight decay.
- **bước và epoch (step / epoch)** — một “bước” là một lần cập nhật tham số dựa trên một lô dữ liệu; một “epoch” là khi mô hình đã đi hết toàn bộ tập huấn luyện một lượt.
- **weight decay** — cơ chế kéo các trọng số về gần 0 sau mỗi bước, nhằm hạn chế mô hình bám quá sát dữ liệu huấn luyện; cách nó được cài đặt là toàn bộ khác biệt giữa Adam và AdamW.
- **fp32, fp16, bf16** — ba định dạng số thực với số bit khác nhau. Điều quan trọng không phải “bao nhiêu bit” mà là bit đó chia thế nào giữa *mũ* (quyết định dải giá trị biểu diễn được) và *định trị* (quyết định độ chính xác).

A. Đặt vấn đề

Giả sử gradient đã tính đúng và không còn chết hay nổ. Câu hỏi còn lại nghe rất đơn giản: biết hướng dốc rồi thì đi bao xa? Nhưng nó khó vì ba lý do chồng lên nhau. Thứ nhất, mặt mất mát bị điều kiện xấu — có hướng dốc gấp trăm lần hướng khác, nên một bước dài theo hướng này là bước ngắn vô vọng theo hướng kia. Thứ hai, gradient bạn có không phải gradient thật mà là ước lượng từ một minibatch, nên nó nhiễu. Thứ ba, mọi thứ phi logic, nên không có lý thuyết nào nói cho bạn bước đi tối ưu.

Toàn bộ lịch sử *optimizer* trong deep learning là câu trả lời cho ba khó khăn ấy, và nó là ví dụ tốt nhất trong tài liệu này về việc *một danh sách thực ra là một mạch lập luận*. Bảy cái tên dưới đây thường được dạy như bảy lựa chọn ngang hàng trong một menu. Chúng không ngang hàng: mỗi cái ra đời để sửa một khiếm khuyết cụ thể của cái trước, và mỗi cái tạo ra một khiếm khuyết mới cho cái sau. Nếu bạn đọc được mạch đó thì bạn không cần nhớ công thức, vì bạn suy lại được

chúng; nếu bạn chỉ nhớ danh sách thì bạn sẽ chọn optimizer theo thói quen và không bao giờ biết mình đang trả giá gì.

TRỰC GIÁC

Momentum thường được kể bằng ẩn dụ hòn bi lăn xuống dốc, nhưng ẩn dụ đó không nói cho bạn biết *vì sao* nó hiệu quả. Cách nhìn đúng hơn với người đã lọc tín hiệu IMU: chuỗi gradient theo thời gian là một tín hiệu, và momentum là một bộ lọc thông thấp một cực đặt lên tín hiệu đó. Trong một rãnh hẹp, thành phần vuông góc với rãnh đổi dấu mỗi bước — đó là thành phần tần số cao, và bộ lọc dập nó. Thành phần dọc theo rãnh giữ nguyên dấu qua nhiều bước — đó là thành phần một chiều, và bộ lọc để nó cộng dồn tới hệ số $1/(1 - \mu)$. Momentum không “thêm quán tính”; nó tách tín hiệu khỏi dao động, đúng như khi bạn lọc bias con quay ra khỏi rung động khung máy.

B. Bảy bước của một mạch lập luận duy nhất (SGD → AdamW)

Trước khi vào mạch, hãy cố định một con số để có cảm giác về mức độ nghiêm trọng. Với một mặt bậc hai có số điều kiện κ , gradient descent hội tụ với hệ số $\left(\frac{\kappa-1}{\kappa+1}\right)$ mỗi bước. Với $\kappa = 1000$, hệ số đó là 0,998, nên cần khoảng 350 bước chỉ để giảm sai số một nửa. Mọi thứ trong mạch dưới đây là những cách khác nhau để không phải trả cái giá đó.

↔ **tiền đề** A/01 — số điều kiện và hệ số hội tụ của gradient descent trên mặt bậc hai là kết quả chuẩn của giải tích số

I Mạch 1 — tiến hoá của optimizer

- 1 Vấn đề** gradient descent ngẫu nhiên đi zigzag trong rãnh hẹp và bò rất chậm theo hướng thoải.

Giải pháp **hạ gradient ngẫu nhiên** (*stochastic gradient descent, SGD*), $\theta \leftarrow \theta - \eta g$, là khởi điểm và cũng là chuẩn so sánh; ưu điểm thật của nó là nhiễu *minibatch* đóng vai trò **chính quy hoá ngầm** (*implicit regularization*).

Giá phải trả bước học η phải nhỏ hơn nghịch đảo độ cong lớn nhất để không phân kỳ, nên nó tự động quá nhỏ cho mọi hướng còn lại.

Vấn đề mới tốc độ bị khoá vào κ , và một giá trị η duy nhất phải phục vụ mọi tham số trong mạng.
- 2 Vấn đề** dao động vuông góc với rãnh làm phí gần hết công.

Giải pháp **momentum**: $v \leftarrow \mu v + g$, $\theta \leftarrow \theta - \eta v$. Thành phần đổi dấu triệt tiêu trong tổng trượt, thành phần cùng dấu cộng dồn tới hệ số $1/(1 - \mu)$, tức gấp 10 lần khi $\mu = 0,9$.

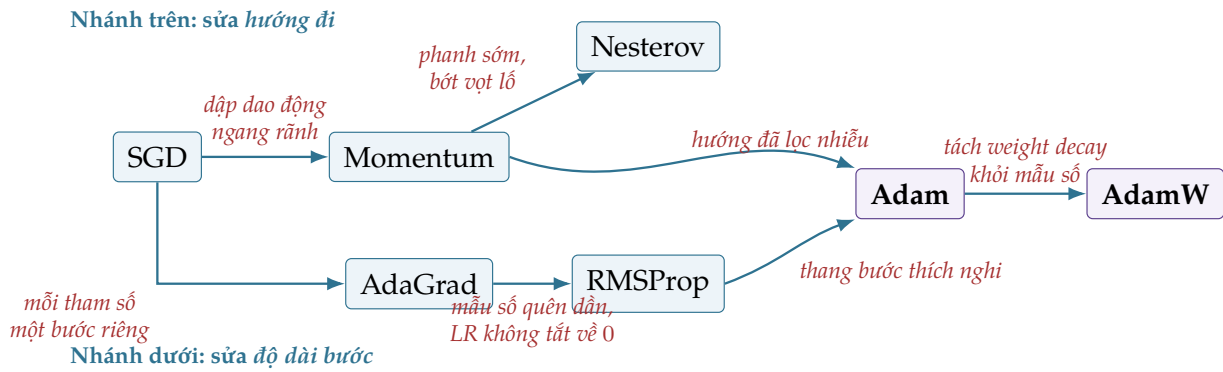
Hệ quả tốc độ hội tụ chuyển từ phụ thuộc κ sang phụ thuộc $\sqrt{\kappa}$, một cải thiện có chứng minh cho trường hợp bậc hai.

Vấn đề mới momentum đã tích lũy không dừng lại được ngay: nghiệm bị vọt lố qua điểm cực tiểu, và trong một rãnh cong thì vọt lố nghĩa là văng ra thành.
- 3 Vấn đề** vọt lố vì gradient được đo ở nơi ta *đang* đứng chứ không phải nơi ta *sắp* tới.

Giải pháp **Nesterov**: tính gradient tại điểm nhìn trước $\theta - \eta \mu v$. Hiệu giữa gradient ở điểm nhìn trước và gradient tại chỗ là một xấp xỉ của số hạng bậc hai. Nó hoạt động như lực hãm tỉ lệ với tốc độ thay đổi của gradient, tức phanh trước khi đâm vào thành chứ không sau.

Giá phải trả thêm một lần bookkeeping, thường được cài bằng phép đổi biến nên chi phí thực tế bằng không.

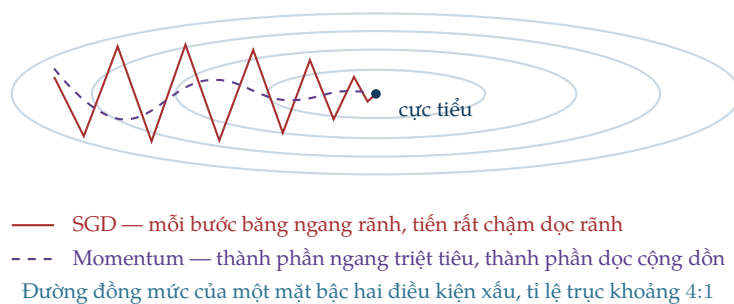
- Vấn đề mới** vẫn còn nguyên vấn đề gốc: một η chung cho tất cả tham số.
- 4 Vấn đề** các tham số có độ lớn gradient lệch nhau hàng bậc, nên không tồn tại một η đúng cho tất cả.
- Giải pháp** **AdaGrad**: tích lũy $G \leftarrow G + g^2$ theo từng tọa độ và bước theo $\eta g / (\sqrt{G} + \varepsilon)$. Tham số hiếm được cập nhật nhận bước lớn, tham số hay được cập nhật nhận bước nhỏ; ý tưởng sinh ra từ các bài toán lỗi thừa và rất hợp lý ở đó.
- Vấn đề mới** G là một tổng đơn điệu tăng, nên bước học hiệu dụng tắt dần về 0. Trong một bài toán lỗi thì đó là tính năng; trong một mạng sâu chạy hàng trăm nghìn bước thì đó là cái chết: mô hình ngừng học trong khi loss còn cao.
- 5 Vấn đề** mẫu số của AdaGrad nhớ quá lâu.
- Giải pháp** **RMSProp**: thay tổng bằng trung bình trượt phân rã, $G \leftarrow \beta G + (1 - \beta)g^2$. Bộ nhớ trở thành hữu hạn với chân trời khoảng $1/(1 - \beta)$ bước, tức chừng 1000 bước khi $\beta = 0,999$, nên bước học hiệu dụng không tắt.
- Vấn đề mới** hướng đi vẫn là gradient thô nên vẫn nhiễu; và trung bình trượt khởi tạo từ 0 nên trong những bước đầu G bị đánh giá thấp, mẫu số quá nhỏ, bước đi quá dài đúng vào lúc mô hình dễ tổn thương nhất.
- 6 Vấn đề** cần cả hai: hướng đã lọc nhiễu và thang bước riêng cho từng tham số.
- Giải pháp** **Adam** [9] giữ hai trung bình trượt: m cho gradient và v cho bình phương gradient. Nó hiệu chỉnh độ chệch bằng $\hat{m} = m/(1 - \beta_1^t)$ và $\hat{v} = v/(1 - \beta_2^t)$, rồi bước theo $\eta \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$. Hiệu chỉnh độ chệch không phải chi tiết trang trí. Ở bước $t = 1$ với $\beta_1 = 0,9$ và $\beta_2 = 0,999$, ta có $m = 0,1g$ và $\sqrt{\hat{v}} = 0,0316|g|$. Bước chưa hiệu chỉnh vì thế dài gấp khoảng 3,2 lần bước mong muốn.
- Giá phải trả** hai trạng thái phụ cho mỗi tham số: với mô hình 100 triệu tham số ở fp32, tổng bộ nhớ là $4P$ cho trọng số, $4P$ cho gradient và $8P$ cho trạng thái, tức 1,6 GB trước khi tính một byte kích hoạt nào.
- Vấn đề mới** hình phạt L2 cộng vào gradient bị chia cho $\sqrt{\hat{v}}$ giống hệt phần còn lại, nên tham số nào có lịch sử gradient lớn thì bị suy giảm ít đi — nó không còn là weight decay theo nghĩa ban đầu nữa.
- 7 Vấn đề** weight decay trong Adam không phải weight decay.
- Giải pháp** **AdamW** [10] tách hẳn hai thứ: $\theta \leftarrow \theta - \eta(\hat{m}/(\sqrt{\hat{v}} + \varepsilon) + \lambda\theta)$, tức phần suy giảm đi thẳng vào trọng số, không qua mẫu số thích nghi.
- Hệ quả** suy giảm trở lại thành một phép co đều, độc lập với độ lớn gradient, và đây là lý do AdamW là mặc định thực tế cho transformer lẫn cho các recipe CNN hiện đại.
- Giá phải trả** λ không còn so sánh được với λ của Adam cũ, nên mọi giá trị chép từ code cũ đều sai thang.
- Vấn đề mới** phần suy giảm nay tỉ lệ với η , nên nó tương tác với lịch học: giảm η theo cosine thì cũng đang giảm dần cường độ chính quy hoá, một hiệu ứng ít người nói ra.



Hình 1.8. Tiến hoá của optimizer không phải một chuỗi thẳng mà là hai nhánh độc lập rồi hợp lại. Nhánh trên sửa *hướng* của bước đi bằng cách lọc nhiễu theo thời gian; nhánh dưới sửa *độ dài* bước riêng cho từng tham số. Adam là chỗ hai nhánh gặp nhau, và AdamW là bản vá cho khiếm khuyết mà chính phép hợp nhất đó sinh ra. Mỗi mũi tên ghi đúng khiếm khuyết mà bước sau gỡ được.

Hình 1.8 nên được nhìn trước khi đọc mạch bảy bước, vì nó cho thấy ngay điều mà đọc tuần tự khó thấy: Adam không phải “bước tiến thứ sáu” mà là phép hợp nhất hai dòng giải quyết hai vấn đề khác nhau, và đó cũng là lý do nó thừa hưởng nhược điểm của cả hai.

Hai nhận xét khép lại mạch. Thứ nhất, hãy để ý rằng không bước nào trong bảy bước trên giải quyết vấn đề phi lỗi; tất cả đều xử lý điều kiện xấu và nhiễu. Optimizer không tìm cực tiểu toàn cục và không hứa điều đó. Thứ hai, cả họ thích nghi có một cái giá chung ít được nêu: chúng *che giấu* một learning rate sai. Adam với η lệch một bậc vẫn cho đường cong loss trông chấp nhận được, trong khi SGD với sai lệch tương tự thì phân kỳ ngay và bạn biết mình sai. Đây là lý do đáng giữ SGD với momentum làm phép thử chẩn đoán, ngay cả khi bản chạy cuối dùng AdamW.



Hình 1.9. Vì sao momentum thắng trong rãnh hẹp. Thành phần vuông góc với rãnh đổi dấu mỗi bước nên triệt tiêu trong tổng trượt; thành phần dọc rãnh giữ nguyên dấu nên cộng dồn. Tỉ lệ trục trong hình chỉ khoảng 4:1 cho dễ nhìn; trong mạng thật số điều kiện lớn hơn hai tới ba bậc, nên hiệu ứng mạnh hơn nhiều.

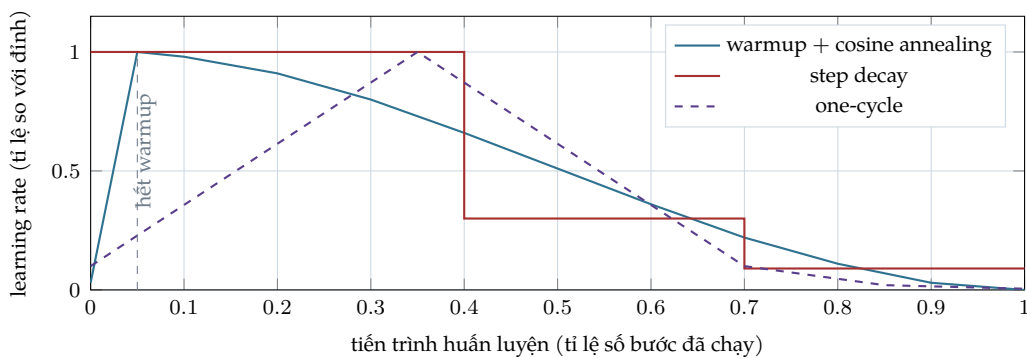
Hình 1.9 là toàn bộ trực giác hình học của bước hai trong mạch, và cũng cho thấy vì sao Nesterov cần thiết: đường đứt nét vẫn vọt qua tâm ở lần tiếp cận đầu tiên nếu μ đủ lớn.

C. Learning rate là siêu tham số quan trọng nhất, không có đối thủ

Nếu chỉ được tune một thứ, tune learning rate. Điều này nghe như khẩu hiệu, nhưng nó có cơ sở. η là đại lượng duy nhất nhân trực tiếp vào mọi bước cập nhật. Sai một bậc ở η đổi hẳn bản chất của quá trình; sai một bậc ở weight decay chỉ làm kết quả kém đi vài phần trăm. Kỹ năng đáng đầu tư nhất là nhận dạng *hình dạng* đường cong học, vì nó tiết kiệm hàng tuần chạy thử: loss tăng vọt hoặc thành NaN trong vài chục bước là η quá cao rõ ràng; loss tụt rất nhanh rồi phẳng ở mức cao và nhiễu mạnh là η hơi cao, hệ đang nảy quanh một lòng chảo mà không xuống được; loss giảm gần như tuyến tính và không có dấu hiệu bão hoà sau nhiều epoch là η quá thấp. Cách

re nhất để định vị là chạy một lượt quét: tăng η theo cấp số nhân trong vài trăm bước và vẽ loss theo η ; vùng dốc nhất trước điểm nổ là vùng đáng dùng.

warmup tăng η tuyến tính từ gần 0 lên giá trị đích, trong vài trăm tới vài nghìn bước. Thoạt nhìn nó là một thủ thuật tùy tiện. Thực ra nó có ít nhất hai lý do rõ ràng, và cả hai đều đã xuất hiện ở trên. Thứ nhất, mẫu số thích nghi của Adam ở những bước đầu được ước lượng từ rất ít mẫu, nên phương sai của $\hat{v}$ rất lớn; một toạ độ vô tình có $\hat{v}$ nhỏ sẽ nhận một bước khổng lồ và phá hỏng số ngay khi chưa có gì để cứu. Thứ hai, thống kê chạy của BatchNorm chưa hội tụ trong vài trăm bước đầu, nên hàm mà bạn đang tối ưu vẫn còn đang thay đổi dưới chân. Warmup mua thời gian cho cả hai bộ ước lượng đó. **Lịch learning rate** thì giải quyết một vấn đề khác: SGD với η cố định không hội tụ về một điểm mà tiến tới một phân bố dừng có phương sai tỉ lệ với η ; giảm η là thu nhỏ bán kính quả cầu mà nghiệm đang lang thang trong đó. Ba lịch phổ biến là step decay, cosine annealing [11] và one-cycle; khác biệt giữa chúng nhỏ hơn nhiều so với khác biệt giữa “có lịch” và “không có lịch”.



Hình 1.10. Ba lịch learning rate trên cùng một trục. Điều hình cho thấy mà lời văn khó nói: chúng khác nhau ở *chỗ* tiêu tiền, không ở tổng lượng. Step decay giữ nguyên bước dài rất lâu rồi cắt đột ngột. Cosine hạ liên tục nên tránh được cú sốc ở mỗi mốc cắt. One-cycle thì cố tình bắt đầu thấp và *tăng* lên đỉnh ở giữa, để đi qua vùng nhiễu nhanh hơn. Đoạn dốc đứng ở đầu đường cosine là warmup: nó tồn tại vì bộ ước lượng $\hat{v}$ của Adam và thống kê BatchNorm đều chưa đáng tin ở vài trăm bước đầu.

Hình 1.10 cũng cho thấy vì sao so sánh hai bài báo dùng hai lịch khác nhau là việc rất dễ sai: hai mô hình cùng số epoch có thể đã trải qua hai quá trình rất khác nhau.

D. Đổi batch size thì đổi gì theo (linear scaling rule)

quy tắc tỉ lệ tuyến tính (*linear scaling rule*) nói rằng nhân batch size lên k lần thì nhân learning rate lên k lần. Quy tắc này đã được kiểm chứng trên ImageNet với warmup đủ dài, tới batch 8192 [12]. Lập luận đơn giản: một bước với batch kB nên tiến xa bằng k bước với batch B , và trung bình của k gradient liên tiếp xấp xỉ gradient của batch gộp khi trọng số chưa đổi nhiều.

Nhưng nguyên nhân sâu hơn mới đáng nhớ, vì nó nói rằng đây không chỉ là chuyện tốc độ. Phương sai của gradient minibatch tỉ lệ nghịch với batch size, nên batch lớn cho gradient sạch hơn — và nhiễu gradient chính là một dạng chính quy hoá ngầm. Batch quá lớn không chỉ “khó tune hơn”: nó *lấy đi* một nguồn chính quy hoá, và các quan sát trên ImageNet cho thấy vượt một ngưỡng nhất định thì độ chính xác kiểm thử tụt dù loss huấn luyện vẫn tốt [13]. Smith và Le đưa ra một cách nhìn thống nhất: giảm η và tăng batch size là hai cách làm cùng một việc, đều là giảm mức nhiễu η/B [14]. Cách nhìn đó giải thích cả linh cảm “tăng batch thì phải tăng LR”, lẫn lý do quy tắc vỡ. Khi η đã lớn tới mức không tăng được nữa vì lý do ổn định, mối liên hệ đứt.

↪ **đối lập với C/12/04** — nhiễu gradient vừa là thứ quy tắc tỉ lệ tuyến tính muốn bảo toàn, vừa là nguồn chính quy hoá ngầm mà batch lớn xoá mất

E. Theo dõi sức khoẻ huấn luyện (training health), vì loss là chỉ báo quá chậm

Đường cong loss cho biết *đã* có vấn đề, thường sau hàng nghìn bước. Ba phép đo dưới đây rẻ như nhau nhưng phát hiện sớm hơn nhiều, vì chúng đo trạng thái bên trong chứ không đo kết quả.

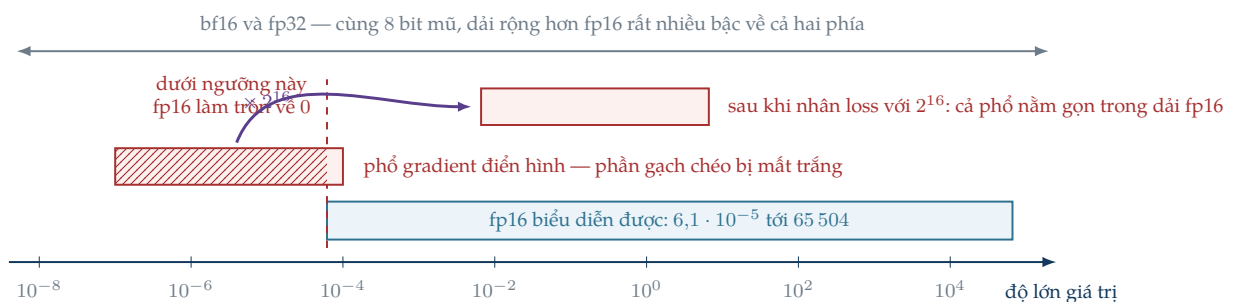
- **Tỉ lệ cập nhật trên trọng số** (*update-to-weight ratio*), theo từng tầng: $\|\eta \Delta \theta\| / \|\theta\|$.
 - Kinh nghiệm thực hành phổ biến: khoảng 10^{-3} là lành mạnh; trên 10^{-2} là mỗi bước làm đổi trọng số vài phần trăm, tức quá lớn; dưới 10^{-4} là tầng đó thực chất đang đứng yên.
 - Vì sao dùng nó thay cho chuẩn gradient thô: nó không thứ nguyên, nên so sánh được giữa các tầng có thang hoàn toàn khác nhau.
- **Phân bố kích hoạt theo chiều sâu** — vẽ độ lệch chuẩn của kích hoạt từng tầng. Co dần theo chiều sâu là vanishing đang hình thành; phình lên là exploding. Đây chính là đại lượng ρ^L của mục trước, quan sát trực tiếp.
- **Tỉ lệ ReLU chết** — phần trăm nơ-ron cho 0 trên toàn bộ một batch xác thực. Vài phần trăm là bình thường; ba mươi phần trăm ở một tầng nghĩa là η đã có lúc quá cao, kể cả khi hiện tại nó đã ổn.

Cả ba chỉ báo chỉ tốn vài dòng hook và nên được ghi lại mặc định, vì giá trị của chúng nằm ở chỗ chúng *đi trước* loss chứ không đi sau.

F. Mixed precision: chuyện của dải động, không phải của độ chính xác

Chuyển sang **fp16** làm training ra NaN, và nguyên nhân thường bị hiểu nhầm là “thiếu độ chính xác”. Thủ phạm thật là *dải động*. fp16 có 5 bit mũ và 10 bit định trị, nên số chuẩn hoá nhỏ nhất là khoảng $6,1 \cdot 10^{-5}$ và số lớn nhất là 65 504. Gradient trong mạng thị giác thường nằm quanh 10^{-7} tới 10^{-4} ; phần lớn phổ đó rơi xuống dưới ngưỡng chuẩn hoá của fp16 và bị làm tròn về 0. Mạng không nổ, nó *tê liệt*: một phần các tầng đơn giản là không nhận được tín hiệu nào.

Lời sửa của **loss scaling** [15] tinh tế ở chỗ nó không đụng vào định dạng số. Nhân loss với một hệ số S cỡ 2^{16} trước khi gọi backward; vì backward tuyến tính theo adjoint đầu vào, mọi gradient bị nhân đúng S lần và cả phổ dịch lên vùng biểu diễn được; trước khi optimizer bước, chia gradient lại cho S . Bản động của kỹ thuật này nhân S lên gấp đôi sau mỗi vài nghìn bước sạch và chia đôi ngay khi phát hiện inf hoặc NaN, bỏ luôn bước đó — tức là nó tự dò biên trên của dải động.



Hình 1.11. Vì sao fp16 làm huấn luyện ra NaN, và vì sao loss scaling chữa được. Trục là độ lớn giá trị theo thang log. Dải fp16 (nền xanh) không bao phủ phần dưới của phổ gradient điển hình, nên phần gạch chéo bị làm tròn về 0 — mạng không nổ mà *tê liệt*. Nhân loss với 2^{16} dịch toàn bộ phổ sang phải mà không đổi hình dạng của nó, đưa cả phổ vào vùng biểu diễn được; chia lại trước bước cập nhật thì không mất gì. bf16 giải bài toán theo hướng khác: nó giữ nguyên số bit mũ của fp32 nên dải đủ rộng từ đầu, đổi lại chỉ còn 7 bit định trị.

Hình 1.11 làm rõ một điểm mà hai chữ “mixed precision” che mất: vấn đề nằm ở *vị trí* của phổ gradient so với dải biểu diễn được, không nằm ở số chữ số có nghĩa. Đó cũng là lý do lời sửa là một phép nhân vô hướng chứ không phải một thuật toán mới.

bf16 giải bài toán theo hướng khác: giữ 8 bit mũ như fp32 nên dải động y hệt fp32 và không cần loss scaling, đổi lại chỉ còn 7 bit định trị.

Chính 7 bit định trị đó dẫn tới điểm kỹ thuật đáng nhớ nhất của cả mục. Độ chính xác tương đối của bf16 là khoảng $2^{-8} \approx 4 \cdot 10^{-3}$. Nhưng ta vừa nói tỉ lệ cập nhật trên trọng số lành mạnh là khoảng 10^{-3} . Nghĩa là bước cập nhật *nhỏ hơn một đơn vị cuối cùng của trọng số*: nếu bạn cộng nó vào một trọng số lưu ở bf16, phép cộng làm tròn về đúng giá trị cũ và bước đi biến mất hoàn toàn. Đây là lý do mọi cài đặt mixed precision nghiêm túc giữ một bản *master weights* ở fp32 cho optimizer, dù forward và backward chạy ở nửa độ chính xác; và cũng là lý do bảng kế toán bộ nhớ không giảm một nửa như người ta hy vọng.

→ cùng nguyên lý với D/20 — lượng tử hoá lúc suy luận hỏi đúng câu này: bit chia thế nào giữa mũ và định trị.

Cuối cùng, **cắt ngưỡng gradient** (*gradient clipping*) theo chuẩn toàn cục là bắt buộc với RNN và thường cần với transformer: nó chặn bước đi khi mặt mất mát có vách dựng đứng, nơi gradient tại một điểm không còn nói gì đúng về hàm ở khoảng cách một bước.

G. Giới hạn và trường hợp hỏng

Linear scaling rule vỡ ở batch lớn, và nó vỡ theo kiểu khó phát hiện: loss huấn luyện vẫn đẹp, chỉ có độ chính xác kiểm thử tụt. Nếu bạn chỉ theo dõi loss huấn luyện, bạn sẽ kết luận rằng batch lớn hoạt động tốt.

Loss scaling che giấu NaN thật. Bộ chia tỉ lệ động bỏ qua mọi bước có tràn số. Giả sử mô hình của bạn sinh NaN vì một lỗi thật, chẳng hạn chia cho 0 hoặc lấy log của số âm. Bạn sẽ thấy một quá trình huấn luyện tưởng như bình thường, nhưng thực chất nó bỏ một tỉ lệ lớn số bước. Đáng ghi lại số bước bị bỏ như một chỉ báo riêng.

Weight decay áp nhầm chỗ là lỗi phổ biến và tốn kém: các tham số γ, β của normalization và các bias không nên bị suy giảm, vì chúng không phải “độ phức tạp” theo nghĩa mà weight decay muốn phạt, và co chúng về 0 làm hỏng chính phép chuẩn hoá. Phần lớn recipe tốt loại các nhóm này ra khỏi decay một cách tường minh.

Cuối cùng, những con số trong mục này thuộc ba mức tin cậy khác nhau và nên được đối xử khác nhau. Hệ số $1/(1 - \mu)$, tỉ lệ 3,2 của bước Adam đầu tiên, và các ngưỡng của fp16 là suy dẫn hoặc tra cứu, nên chắc chắn. Kết quả linear scaling tới batch 8192 là một phép đo đã công bố trên một cấu hình cụ thể, không phải một định luật. Còn ngưỡng 10^{-3} cho tỉ lệ cập nhật trên trọng số chỉ là kinh nghiệm thực hành, được truyền lại rộng rãi. Tôi chưa thấy nghiên cứu hệ thống nào xác định lại nó cho các kiến trúc hiện đại. Hãy dùng nó như một cờ báo động, đừng dùng như mục tiêu để tối ưu.

CẠM BÃY

`torch.optim.Adam(..., weight_decay= $\lambda$ )` *không* phải AdamW: nó cộng $\lambda\theta$ vào gradient rồi cho cả cục đi qua mẫu số thích nghi, đúng cái lỗi mà AdamW sinh ra để sửa. Muốn weight decay tách rời thì phải dùng AdamW. Đây là lỗi mà rất nhiều code cũ vẫn mắc, và nó đặc biệt khó phát hiện. Hậu quả không phải một exception mà là điểm số thấp hơn vài phần trăm. Mức đó vừa đủ nhỏ để bạn đổ cho kiến trúc, vừa đủ lớn để mất một vị trí trên bảng xếp hạng.

CẦN NHỚ

Optimizer chỉ định hình lại bước đi; nó không cứu được một learning rate sai. Thứ tự đầu tư công sức: learning rate và lịch của nó trước, batch size cùng quy tắc tỉ lệ sau, chọn

optimizer cuối cùng. Bảy cái tên trong mạch không phải bảy lựa chọn, mà là bảy lần vá liên tiếp trên cùng một vấn đề. [T2]

H. Kết nối

Mục này giả định gradient đã lành mạnh, tức giả định toàn bộ [C/12/02-lam-cho-training-chay-duoc](#). Nó chuyển tiếp sang [C/12/04-tong-quat-hoa-va-chan-doan](#). Ở đó, nhiều gradient của quy tắc tỉ lệ tuyến tính được nhìn lại như một công cụ chính quy hoá. Cũng ở đó, bảng kế toán bộ nhớ của trạng thái Adam trở thành lý do tồn tại của ZeRO. Phần dài dòng của fp16 và bf16 là tiền đề trực tiếp cho [D/20-toi-uu-va-nen-mo-hinh](#) (lượng tử hoá), nơi cùng một câu hỏi được đặt cho suy luận thay vì cho huấn luyện, và cho [B/11-gioi-han-tinh-toan](#) về phía băng thông. Nếu bạn muốn thấy các lựa chọn trong mục này ảnh hưởng thế nào tới kết luận của một paper, [E/24-thi-nghiem-va-ablation](#) là chương nói rằng phần lớn “cải tiến kiến trúc” hoá ra là cải tiến lịch học.

▮ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Momentum sửa được gì mà SGD không làm được, và nó tạo ra vấn đề mới nào buộc Nesterov phải ra đời?
2. AdaGrad và RMSProp khác nhau đúng một phép toán. Phép toán đó là gì, và vì sao khác biệt nhỏ ấy lại là ranh giới giữa “dùng được cho mạng sâu” và “không dùng được”?
3. Vì sao AdamW khác Adam cộng L2, và bạn kỳ vọng khác biệt thể hiện ở đại lượng đo được nào?
4. Vì sao warmup cần thiết với Adam hơn là với SGD, nếu trả lời bằng tính chất của bộ ước lượng $\hat{v}$?
5. Bạn tăng batch size gấp 8 và nhân learning rate lên 8; loss huấn luyện tốt hơn nhưng accuracy kiểm thử tụt 1 phần trăm — hai cách giải thích là gì, và phép đo nào phân biệt chúng?
6. Vì sao huấn luyện ở bf16 vẫn cần giữ trọng số fp32, dù bf16 có dải động bằng fp32?
7. Đường cong học giảm nhanh rồi phẳng ở mức cao — ba nguyên nhân khả dĩ là gì, và ba phép đo nào phân biệt chúng?

Nguồn

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.

1.5 Tổng quát hoá và chẩn đoán (Generalization and Diagnosis)

PHỤ THUỘC C/12/03-toi-uu-hoa (nhiều gradient, kế toán bộ nhớ của trạng thái Adam); B/04-nen-tang-hoc-tu-du-lieu và B/07-chia-du-lieu-va-danh-gia (vì sao “tổng quát hoá” chỉ có nghĩa khi tập chia đúng).

TÓM TẮT

Mục này xử lý khoảng cách giữa “mạng khớp được dữ liệu huấn luyện” và “mạng dùng được”, rồi biến phần còn lại thành quy trình chẩn đoán. Đọc xong bạn xếp được các công cụ **chính quy hoá** (*regularization*) theo hiệu quả thực tế trong thị giác, thay vì theo độ nổi tiếng. Bạn biết đưa mô hình lên nhiều GPU theo đúng thứ tự tăng dần độ phức tạp. Và bạn có một quy trình bắt lỗi cho kết luận trong vài phút thay vì vài ngày. Nếu chỉ nhớ một điều: trước khi chạy full, hãy *overfit một batch 20–50 mẫu tới loss gần 0*. Phép thử này tách bạch “vòng lặp huấn luyện có bug” khỏi “mô hình không tổng quát hoá được”. Phần lớn thời gian bị mất là do người ta nhầm hai thứ đó với nhau.

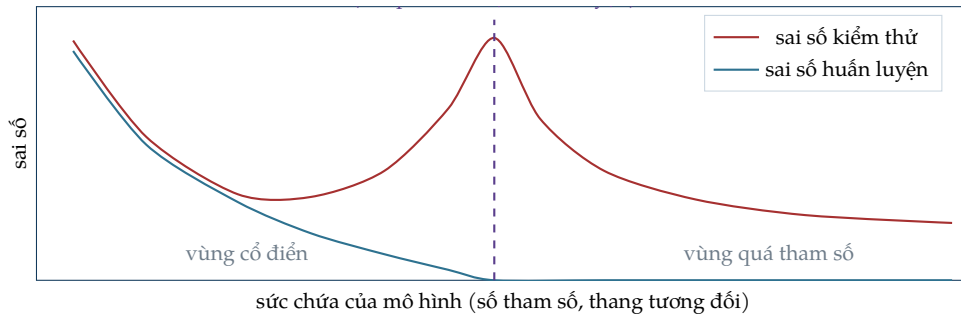
KIẾN THỨC CẦN CÓ TRƯỚC

- **tập huấn luyện, xác thực, kiểm thử** — ba phần rời nhau của dữ liệu: phần để học, phần để chọn siêu tham số và dùng đúng lúc, phần chỉ dùng đúng một lần để báo cáo. Trộn lẫn ba phần là cách phổ biến nhất để tự lừa mình.
- **overfitting** — mô hình khớp rất tốt dữ liệu huấn luyện nhưng kém trên dữ liệu mới, vì nó đã học cả những chi tiết chỉ đúng với tập huấn luyện.
- **chính quy hoá (regularization)** — mọi kỹ thuật nhằm thu hẹp khoảng cách giữa kết quả huấn luyện và kết quả thực tế — bằng cách ràng buộc mô hình, thêm nhiễu, hoặc thêm thông tin.
- **augmentation** — sinh thêm mẫu huấn luyện bằng cách biến đổi mẫu có sẵn (lật, cắt, đổi màu); mỗi phép biến đổi là một tuyên bố “nhân không đổi dưới phép này”.
- **sức chứa (capacity)** — độ phong phú của lớp hàm mà mô hình biểu diễn được, thường xấp xỉ bằng số tham số; lý thuyết cổ điển gắn sức chứa với overfitting, còn thực nghiệm hiện đại thì không.
- **rò rỉ dữ liệu (data leakage)** — thông tin của tập đánh giá lọt vào quá trình huấn luyện, khiến điểm số đẹp giả tạo — ví dụ hai khung hình liên tiếp của cùng một video nằm ở hai bên ranh giới chia tập.
- **tiền trình và GPU** — trong huấn luyện phân tán, mỗi GPU thường chạy một tiến trình riêng giữ một bản mô hình; “đồng bộ” nghĩa là các tiến trình trao đổi gradient để giữ các bản đó giống nhau.
- **EMA (trung bình trượt lũy thừa)** — một bản trọng số phụ được cập nhật bằng trung bình có trọng số giảm dần theo thời gian; nó mượt hơn trọng số hiện thời và thường cho kết quả đánh giá tốt hơn.

A. Đặt vấn đề

Loss huấn luyện xuống đẹp, độ chính xác trên tập kiểm thử thì không. Đây là bài toán cũ nhất của học máy, nhưng cách hiểu nó trong deep learning đã thay đổi hẳn và việc mang nguyên trực giác cũ sang là một nguồn suy luận sai lớn. Trực giác cổ điển nói: mô hình quá nhiều tham số so với dữ liệu thì học thuộc lòng, nên hãy giảm sức chứa mô hình. Nhưng một mạng hiện đại khớp hoàn hảo được cả một tập ImageNet với nhãn đã bị xáo ngẫu nhiên [16]. Sức chứa của nó dư sức học thuộc. Vậy mà chính nó vẫn tổng quát hoá tốt khi nhãn là chân trị. Các cận sai số dựa trên

sức chứa do đó gần như rỗng, và hiện tượng *double descent* còn cho thấy tăng sức chứa vượt qua điểm nội suy có thể làm sai số kiểm thử *giảm* trở lại [17], [18].



Hình 1.12. Hiện tượng giảm kép (*double descent*), vẽ định tính. Trực giác cổ điển chỉ mô tả đúng nửa trái: sai số kiểm thử đi xuống rồi đi lên khi mô hình lớn dần. Nhưng ngay tại ngưỡng nội suy — chỗ mô hình vừa đủ sức khớp hết dữ liệu huấn luyện — sai số kiểm thử đạt *đỉnh* rồi *giảm trở lại* khi ta tiếp tục tăng sức chứa. Đây là lý do câu “mô hình quá lớn thì sẽ overfit” không dùng được làm quy tắc, và là lý do phải chọn công cụ chính quy hoá theo đo đặc chứ không theo lý thuyết sức chứa. Hình là minh hoạ định tính; vị trí đỉnh và độ sâu của lần giảm thứ hai phụ thuộc tác vụ và lượng nhiễu nhân.

Hình 1.12 nói rõ điều mà một câu trích dẫn không nói được: hai trực giác đối nghịch nhau đều đúng, chỉ là đúng ở hai nửa khác nhau của cùng một trục.

Hệ quả thực hành rất cụ thể: đừng chọn công cụ theo lý thuyết sức chứa, hãy chọn theo hiệu quả đo được trong đúng lĩnh vực của bạn. Và vì phần lớn các lần “mô hình không tổng quát hoá” thực ra là “vòng lặp huấn luyện có lỗi”, phần thứ hai của mục này quan trọng không kém phần thứ nhất. Giá trị cụ thể: sau mục này bạn có một cây quyết định để chẩn đoán một lần chạy hỏng, và một bảng triệu chứng để tra thay vì đoán.

→ **tiền đề** B/07 — khoảng cách train-val chỉ có nghĩa nếu tập được chia đúng; chia sai thì mọi kết luận ở đây sụp.

TRỰC GIÁC

Chính quy hoá thường được kể như “ràng buộc mô hình cho nó bớt phức tạp”. Nhưng công cụ mạnh nhất trong thị giác lại là augmentation, và nó không ràng buộc gì cả: nó *thêm thông tin*. Hãy nghĩ theo kiểu người hiệu chuẩn cảm biến: khi bạn lắc camera qua nhiều tư thế lúc hiệu chuẩn, bạn không làm mô hình camera đơn giản đi, bạn đang nói cho bộ ước lượng biết đại lượng nào *không được phép* thay đổi kết quả. Augmentation làm đúng việc đó với mạng: mỗi phép biến đổi là một câu tuyên bố “nhân không đổi dưới phép này”. Vì thế nó thắng các công cụ ràng buộc sức chứa — nó trả lời một câu hỏi mà chúng không trả lời được.

B. Bộ công cụ chính quy hoá, xếp theo hiệu quả thực tế trong thị giác

Thứ tự dưới đây là thứ tự *hiệu quả thực hành trong computer vision*, không phải thứ tự nổi tiếng. Nó khác đáng kể thứ tự mà sách giáo khoa trình bày.

- **Augmentation** — gần như luôn cho lợi ích lớn nhất trên mỗi giờ công bỏ ra, vì nó là công cụ duy nhất trong danh sách *thêm thông tin* thay vì lấy bớt tự do. Chi tiết ở mục C.
- **Early stopping** — dừng khi loss xác thực bắt đầu xấu. Gần như miễn phí, không có siêu tham số ngoài kiên nhẫn, và trong nhiều recipe nó là thứ đang âm thầm gánh phần lớn công việc chính quy hoá.
- **Weight decay** — vẫn hữu ích, nhưng cơ chế thật của nó trong mạng có normalization không phải cơ chế trong sách. Với một tầng conv đứng ngay trước BatchNorm, nhân W với một

hằng số *không đổi* hàm số mà mạng biểu diễn, vì BN chuẩn hoá lại ngay sau đó. Vậy weight decay không thu hẹp lớp hàm; cái nó thật sự điều khiển là *độ lớn của bước học hiệu dụng*, vì bước tương đối $\|\Delta W\|/\|W\|$ tăng khi $\|W\|$ bị co lại. Đây là lý do weight decay và learning rate không độc lập với nhau như tên gọi gợi ý.

- **EMA của trọng số** — giữ một bản trung bình trượt của trọng số và dùng nó để đánh giá. Rẻ, gần như luôn cải thiện một chút, và cơ chế thì đã nói ở mục trước: SGD ở cuối quá trình lang thang trong một quả cầu bán kính tỉ lệ với η , nên lấy trung bình là đứng vào tâm quả cầu đó thay vì một điểm ngẫu nhiên trên nó.
- **Stochastic depth** — bỏ ngẫu nhiên cả một khối residual trong lúc huấn luyện [19]. Hợp với mạng rất sâu; nó là “dropout ở cấp khối” và không phá thống kê không gian như dropout cấp phần tử.
- **Dropout** — *đã mất phần lớn vai trò* trong CNN thị giác hiện đại [20]. Có hai lý do. BatchNorm đã bơm sẵn nhiễu qua thống kê batch, nên nhiễu thêm là thừa. Và hai kỹ thuật xung khắc nhau: dropout làm phương sai kích hoạt lúc huấn luyện khác lúc suy luận, đúng đại lượng mà BN đang cố cố định. Nó vẫn sống khỏe ở tầng đầu ra của transformer và ở các mô hình ngôn ngữ.

Điểm đáng nhớ của cả danh sách không phải thứ tự mà là *vì sao* thứ tự đó lệch với sách vở: sách xếp theo mức độ ràng buộc lớp hàm, thực tế xếp theo lượng thông tin mà công cụ mang thêm vào hoặc lượng nhiễu mà nó khớp với chế độ suy luận.

C. Augmentation là nơi bạn phát biểu invariance

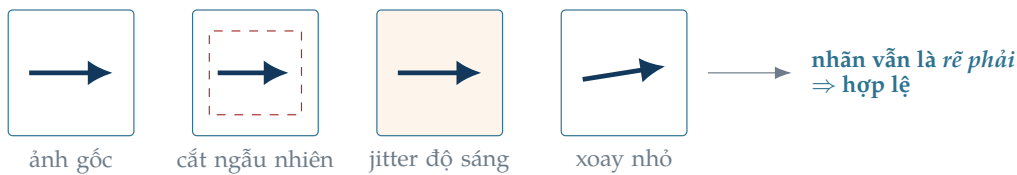
Câu hỏi “augmentation nào là đúng” không có câu trả lời chung, nhưng có một cách nhìn thống nhất làm nó trở thành câu hỏi trả lời được: *mỗi phép augmentation là một tuyên bố về tính bất biến mà bạn muốn mô hình có*. Lật ngang là tuyên bố “lớp không đổi khi lật trái phải”. Jitter màu là tuyên bố về bất biến với điều kiện chiếu sáng. Cắt ngẫu nhiên là tuyên bố “đối tượng có thể ở bất kỳ đâu và bất kỳ tỉ lệ nào”. Từ đó suy ra ngay quy tắc chọn: hãy đưa vào đúng những phép biến đổi thật sự xảy ra trong dữ liệu triển khai của bạn, và loại bỏ những phép phá huỷ nhãn.

Chính vì vậy augmentation có ý nghĩa vật lý mạnh hơn augmentation ngẫu nhiên. Nếu camera của bạn có nhiều đọc, nhiều bắn photon và một đường ống ISP cụ thể, thì mô phỏng đúng chuỗi đó (xem A/02-vat-ly-tao-anh) tạo ra các mẫu nằm trên đúng đa tạp mà mô hình sẽ gặp; còn cộng nhiễu Gauss vào ảnh đã qua ISP thì tạo ra các mẫu không tồn tại trong thực tế.

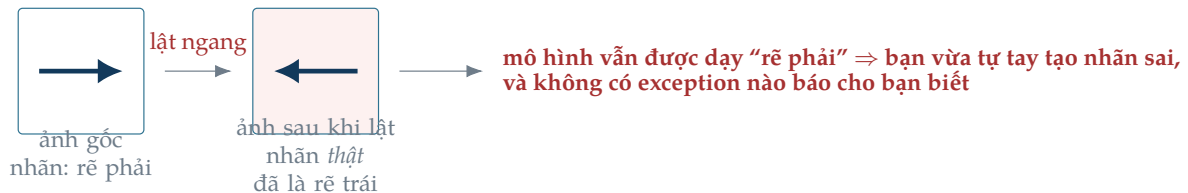
↔ **hệ quả của A/02** — chuỗi tạo ảnh vật lý quyết định phép biến đổi nào sinh ra mẫu có thật, phép nào sinh ra mẫu không bao giờ gặp.

Nhóm hiện đại thì mở rộng ý tưởng theo hai hướng khác nhau: **Mixup** và **CutMix** trộn hai mẫu và trộn cả nhãn theo cùng tỉ lệ [21], [22], tức tuyên bố một giả định về *tính tuyến tính giữa các lớp* chứ không còn về bất biến hình học; **RandAugment** thì tự động hoá việc chọn cường độ và số phép [23], giảm không gian tune xuống hai con số; còn copy-paste và mosaic ghép các đối tượng vào bối cảnh mới, đặc biệt hiệu quả cho detection vì chúng tăng số đối tượng mỗi ảnh.

Giữ nhãn — mỗi phép này là một tuyên bố “lớp không đổi dưới phép biến đổi này”



Phá nhãn — cùng một phép, sai bài toán



Hình 1.13. Augmentation là nơi bạn phát biểu tính bất biến, nên nó chỉ đúng khi tuyên bố đó đúng. Hàng trên: bốn phép biến đổi không đụng tới thứ quyết định nhãn, nên chúng dạy mô hình bỏ qua các chiều biến thiên vô nghĩa. Hàng dưới: cùng một phép lật ngang, nhưng ở bài toán mà *hướng* chính là lớp, nó tạo ra một cặp ảnh–nhãn mâu thuẫn. Đây cũng là lý do việc chọn augmentation phải bắt đầu từ câu hỏi “cái gì được phép đổi mà nhãn không đổi”, chứ không phải từ danh sách mặc định của thư viện.

Hình 1.13 cho thấy vì sao không tồn tại một bộ augmentation “tốt” chung: cùng một phép biến đổi vừa là công cụ mạnh nhất vừa là nguồn nhãn sai, tùy vào bài toán.

Chỗ hỏng thì rất cụ thể và đáng thuộc: augmentation phá nhãn. Lật ngang một ảnh chữ số hoặc biến báo rẽ trái là tạo ra một mẫu có nhãn sai. Jitter màu trong bài toán mà màu *chính là* lớp (đèn giao thông, phân loại quả chín) cũng vậy. Xoay lớn trong ảnh trên cao thì hợp lý, trong ảnh chụp bởi camera gắn xe thì không, vì trọng lực là một tiên nghiệm thật.

D. Từ một GPU tới nhiều GPU

Bốn bước dưới đây xếp theo đúng thứ tự tăng dần độ phức tạp và mức tiết kiệm bộ nhớ. Nguyên tắc là *chỉ leo lên bước sau khi bước trước đã hết dư địa*. Trước khi đọc bảng, hãy cố định bảng kế toán. Với P tham số huấn luyện ở fp32 bằng Adam, mỗi GPU phải giữ $4P$ byte trọng số, $4P$ byte gradient và $8P$ byte trạng thái Adam. Tổng là $16P$ byte, tức 4,8 GB khi $P = 300$ triệu — trước khi tính một byte kích hoạt nào.

Bảng 1.5. Bốn mức song song hoá dữ liệu. Cột bộ nhớ tính cho N tiến trình và bỏ qua kích hoạt; cột cuối là cột đáng đọc kỹ nhất.

Bước	Cơ chế	Bộ nhớ mỗi GPU	Hỏng khi nào
DataParallel	Một tiến trình, phát tán mô hình sang các GPU mỗi bước	$16P$	Nghẽn ở GIL và ở việc sao chép mô hình; GPU 0 quá tải. Coi như đã lỗi thời
DDP	Mỗi GPU một tiến trình, all-reduce gradient sau backward	$16P$	Seed và sampler không đặt đúng thì các tiến trình thấy cùng dữ liệu; BatchNorm không đồng bộ nên mỗi GPU là một mô hình hơi khác
Gradient accumulation	Chia batch thành micro-batch, cộng dồn gradient rồi mới bước	$16P$, nhưng kích hoạt giảm theo micro-batch	Không giảm bộ nhớ trạng thái; thống kê BatchNorm vẫn tính theo micro-batch nên batch hiệu dụng lớn mà thống kê thì không
ZeRO / FSDP [24]	Chia nhỏ trạng thái (giai đoạn 1), gradient (2), rồi cả trọng số (3) giữa các tiến trình	$8P + 8P/N$ tới $16P/N$	Lưu lượng giao tiếp tăng theo giai đoạn; nghẽn khi mạng liên kết chậm hoặc mô hình nhiều tầng nhỏ

Ba cạm bẫy trong Bảng 1.5 đáng nêu tên riêng vì chúng không gây lỗi mà chỉ gây kết quả kém. Thứ nhất là *seed và sampler*: nếu mọi tiến trình dùng cùng seed cho việc xáo dữ liệu, bạn không huấn luyện trên N lần dữ liệu mà trên cùng một dòng dữ liệu N lần. Thứ hai là *BatchNorm*. Mỗi tiến trình tính thống kê trên batch cục bộ của nó. Với detection chạy một tới hai ảnh mỗi GPU, thống kê nhiều tới mức vô dụng. Đây chính là lý do GroupNorm và SyncBN tồn tại, và cũng là vấn đề đã gặp ở mục 2 của chương này. Thứ ba là *cách gộp loss*: lấy tổng thay vì trung bình làm learning rate hiệu dụng bị nhân với số tiến trình, một lỗi im lặng có hậu quả đúng bằng việc đặt sai learning rate một bậc.

↪ **hệ quả của C/12/03** — bảng kế toán $16P$ byte cho trọng số, gradient và trạng thái Adam là lý do tồn tại của ZeRO

Song song với bốn bước trên còn một trục thứ hai đã nhắc ở mục 1: **gradient checkpointing** [2] đổi khoảng 30 phần trăm thời gian tính lấy việc giảm bộ nhớ kích hoạt từ $O(L)$ xuống $O(\sqrt{L})$. Trong thị giác, kích hoạt thường lớn hơn trọng số rất nhiều, nên với một GPU đơn thì checkpointing hầu như luôn là bước nên thử trước ZeRO.

E. Chính quy hoá ngầm: chỗ ngành chưa có câu trả lời

Cần biết mục này để *không suy luận sai*, chứ không phải để dùng. Câu hỏi mở là: vì sao một mạng quá tham số, đủ sức học thuộc nhiều, lại chọn đúng nghiệm tổng quát hoá được khi ta huấn luyện nó bằng SGD? Các mảnh câu trả lời hiện có đều là mảnh: nhiều của SGD thiên vị các nghiệm ở vùng phẳng của mặt mất mát; các thuật toán bậc nhất có xu hướng tìm nghiệm chuẩn nhỏ trong các bài toán tuyến tính quá tham số; khởi tạo và lịch học đều để lại dấu vết trên nghiệm cuối. Chưa có lý thuyết nào ghép các mảnh đó lại thành một lời giải thích đầy đủ cho mạng sâu thật.

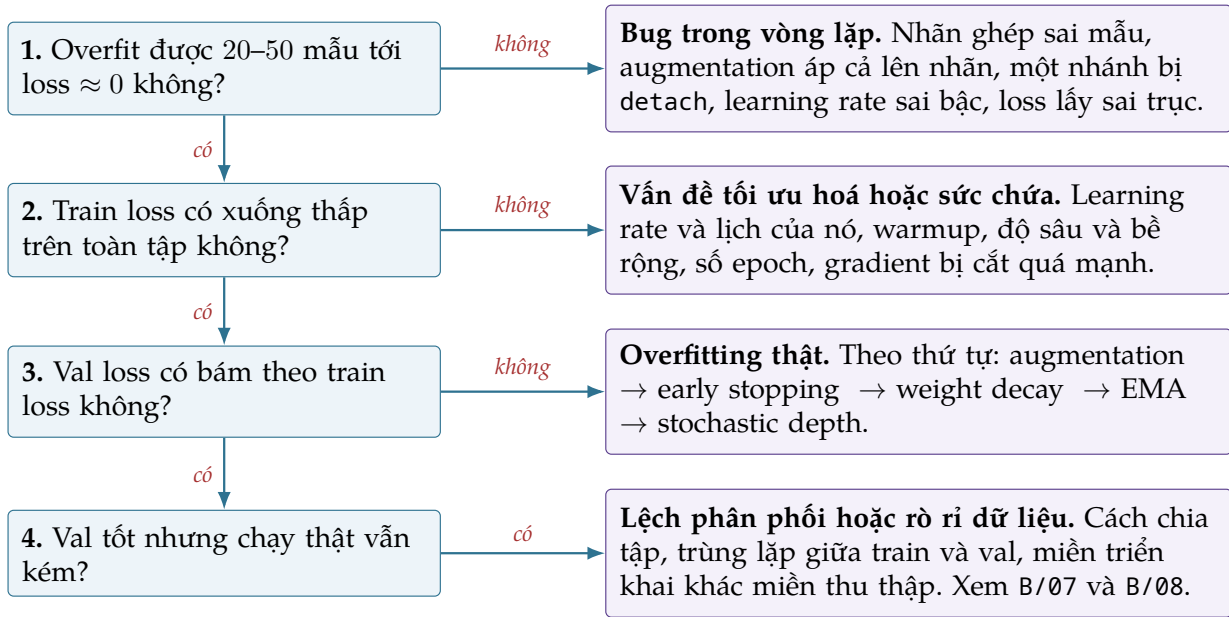
Hệ quả cho cách bạn đọc và nói: đừng kết luận “mô hình này quá lớn nên sẽ overfit” như một mệnh đề chắc chắn, vì nó thường sai trên thực nghiệm; và cũng đừng kết luận ngược lại rằng sức chứa không quan trọng. Phát biểu trung thực là: quá trình huấn luyện, chứ không chỉ lớp hàm, quyết định nghiệm — và ta chưa mô tả được “chứ không chỉ” đó một cách định lượng.

F. Phản xạ debug số một: overfit một batch nhỏ

Trước khi chạy full, lấy 20 tới 50 mẫu, tắt mọi augmentation và mọi chính quy hoá, rồi huấn luyện tới khi loss huấn luyện gần 0. Nếu không đạt được điều đó, dừng lại: bug nằm trong vòng lặp, không nằm ở khả năng tổng quát hoá, và mọi phút bỏ ra để tune siêu tham số lúc này là phí.

Sức mạnh của phép thử nằm ở chỗ nó *loại bỏ đúng các biến gây nhiễu*. Trên 20 mẫu, một mạng có hàng triệu tham số thừa sức nhớ hết, nên nếu nó không nhớ được thì tín hiệu không đi từ dữ liệu tới loss hoặc gradient không đi ngược lại được. Nó kiểm tra toàn tuyến — đọc dữ liệu, tiền xử lý, ghép nhãn, hàm mất mát, backward, bước cập nhật — trong vài phút, thay vì để bạn suy đoán từ một đường cong sau ba giờ. Đây cũng là lý do nó phải là việc đầu tiên, không phải việc cuối cùng.

Hình 1.14 biến phép thử đó thành một cây quyết định đầy đủ. Điểm đáng chú ý là mỗi nhánh dẫn tới một *họ* nguyên nhân khác hẳn nhau, nên trả lời sai câu hỏi đầu tiên khiến bạn đi tìm ở nhầm họ suốt phần còn lại.



Hình 1.14. Cây chẩn đoán một lần chạy không đạt kỳ vọng. Bốn câu hỏi phải hỏi theo đúng thứ tự này, vì mỗi câu chỉ có nghĩa khi câu trước đã trả lời “có”; bỏ qua câu số một là nguyên nhân phổ biến nhất của việc tune siêu tham số cho một hệ thống đang có bug.

G. Phá hỏng có chủ đích: xây thư viện triệu chứng

Kỹ năng cao nhất của cả chương không phải biết cách sửa mà là biết *nhìn một đường cong và đoán ra nguyên nhân*. Kỹ năng đó không đến từ đọc; nó đến từ việc cố ý làm hỏng từng thứ một và ghi lại hình dạng đường cong tương ứng. Bảng 1.6 là tám phiên bản hỏng đáng chạy, và cột cuối là cột biến bảng này từ một danh sách thành một công cụ chẩn đoán.

Bảng 1.6. Tám lỗi cố ý và triệu chứng của chúng. Chạy cả tám, vẽ tám đường cong vào một hình duy nhất, và giữ hình đó.

Phá hỏng	Triệu chứng trên đường cong	Phân biệt bằng phép đo nào
Learning rate $\times 100$	Loss vọt lên rồi thành NaN trong vài chục bước, hoặc phẳng ngay ở mức phỏng đoán ngẫu nhiên	Tỉ lệ update trên trọng số ở bước đầu vượt 10^{-1} ; chuẩn gradient tăng theo bước
Learning rate $\div 100$	Loss giảm gần như tuyến tính, không có dấu hiệu bão hoà khi hết lịch	Tỉ lệ update trên trọng số dưới 10^{-5}
Quên shuffle	Loss răng cưa với chu kỳ đúng bằng một epoch	Vẽ loss theo từng bước thay vì theo epoch; hình răng cưa lộ ra ngay
Quên eval()	Val loss thấp bất thường và <i>đổi</i> khi đổi batch size lúc đánh giá	Chạy val hai lần với batch 1 và batch 64, so kết quả
Nhãn lệch một chỉ số	Train loss giảm bình thường; val accuracy quanh mức ngẫu nhiên	Ma trận nhầm lẫn lệch hẳn khỏi đường chéo một cột
Quên chuẩn hoá đầu vào	Hội tụ chậm hơn rõ rệt; nếu có BatchNorm thì gần như không thấy gì	Histogram của batch đầu vào; thống kê kích hoạt tầng đầu
Khởi tạo toàn 0	Loss đứng đúng mức ngẫu nhiên, không nhích	Các hàng của W giống hệt nhau sau vài bước: đối xứng chưa bị phá
Dropout 0,9	Train loss <i>cao hơn</i> val loss; cả hai hội tụ rất chậm	Khoảng cách train trừ val mang dấu âm — điều gần như không bao giờ xảy ra khi mọi thứ đúng

H. Giới hạn và trường hợp hỏng

Phép thử “overfit một batch nhỏ” có một chế độ hỏng đáng biết, và nó là loại chế độ hỏng khó chịu nhất: phép thử *qua* trong khi hệ thống vẫn sai. Nếu nhãn của bạn bị hoán vị theo một quy tắc cố định — ví dụ chỉ số lệch một — thì trên 20 mẫu, mạng học thuộc ánh xạ sai đó dễ dàng và loss vẫn về 0. Cùng lý do, một bug trong tiền xử lý áp giống nhau cho mọi mẫu cũng không bị bắt. Nói cách khác, phép thử chứng minh “tín hiệu chảy được từ đầu vào tới loss và ngược lại”, không chứng minh “tín hiệu chảy đúng thứ”. Bổ sung rẻ nhất là nhìn trực tiếp vài chục mẫu đã qua toàn bộ pipeline, kèm nhãn, ngay trước khi vào mô hình.

↔ **tiền đề** E/24 — ablation chạy trên một hệ thống còn bug là vô nghĩa, nên phép thử này phải đứng trước mọi thí nghiệm so sánh

Bảng triệu chứng cũng có giới hạn: nó được lập trên một cấu hình cụ thể, và một số triệu chứng bị normalization che mất — “quên chuẩn hoá đầu vào” gần như vô hình khi mạng có BatchNorm ở tầng đầu. Đây là mặt trái đã nói ở mục 2: normalization làm hệ thống bền hơn với lỗi và vì thế xoá bớt các triệu chứng mà bạn muốn học đọc. Cuối cùng, thứ tự ưu tiên các công cụ chính quy hoá ở mục B chỉ là *kinh nghiệm thực hành trong thị giác*. Nó được tổng hợp từ các recipe phổ biến, không phải từ một nghiên cứu so sánh có kiểm soát. Ở lĩnh vực khác, chẳng hạn mô hình ngôn ngữ, thứ tự đó khác.

CẠM BẮY

Rõ rĩ dữ liệu là cách phổ biến nhất để có một đường cong xác thực đẹp và một hệ thống vô dụng, và nó gần như không bao giờ trông giống một lỗi. Ba dạng hay gặp trong thị giác: chia tập theo khung hình thay vì theo chuỗi, khiến hai khung hình cách nhau 1/30 giây nằm ở hai bên ranh giới; augmentation hoặc chuẩn hoá được tính trên *toàn bộ* dữ liệu trước khi chia; và các ảnh trùng lặp hoặc gần trùng nằm ở cả hai phía. Dấu hiệu nhận biết: khoảng cách train–val nhỏ bất thường, và điểm số tụt mạnh ngay khi bạn đánh giá trên dữ liệu thu ở một buổi khác.

CẦN NHỚ

Trước khi tune bất cứ thứ gì, overfit 20–50 mẫu tới loss gần 0. Đây là ranh giới tách “vòng lặp có bug” khỏi “mô hình không tổng quát hoá được”. Hai họ nguyên nhân đó cần hai bộ công cụ hoàn toàn khác nhau. Nhắm chúng với nhau là cách tốn thời gian nhất trong nghề. [T1]

I. Kết nối

Mục này khép lại chương 12 và mở sang hai hướng. Về phía dữ liệu, “tổng quát hoá” chỉ có nghĩa khi tập được chia đúng, nên B/07-chia-du-lieu-va-danh-gia là tiền đề bắt buộc của cả mục, và B/08-dich-chuyen-phan-phoi là chương giải thích nhánh cuối cùng của cây chẩn đoán. Về phía kiến trúc, thư viện triệu chứng ở mục G là công cụ bạn sẽ dùng liên tục khi đọc C/13-kien-truc-backbone. Bài học lớn nhất của chương đó là ConvNeXt: phần lớn chênh lệch giữa các kiến trúc hoá ra là chênh lệch công thức huấn luyện. Bài học ấy chỉ có nghĩa nếu bạn phân biệt được hai nguồn. Cuối cùng, sự nghi ngờ có kỷ luật với một bảng so sánh là nội dung của E/24-thi-nghiem-va-ablation.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao lập luận “mô hình nhiều tham số hơn số mẫu thì sẽ học thuộc lòng” không còn dùng được, và điều đó thay đổi cách bạn chọn công cụ chính quy hoá thể nào?
2. Trong một mạng có BatchNorm, weight decay thật sự điều khiển đại lượng nào? Vì sao nó không thu hẹp lớp hàm?
3. Augmentation nào bạn phải *bỏ* nếu bài toán là phân loại đèn giao thông, và vì sao?
4. Gradient accumulation cho batch hiệu dụng lớn hơn nhưng không sửa được một vấn đề của BatchNorm. Vấn đề đó là gì?
5. Phép thử “overfit một batch nhỏ” qua được nhưng hệ thống vẫn sai — tình huống cụ thể nào gây ra chuyện đó, và phép kiểm tra bổ sung nào bắt được nó?
6. Bạn thấy train loss cao hơn val loss — hai nguyên nhân khả dĩ là gì?
7. ZeRO giai đoạn 3 giảm bộ nhớ xuống $16P/N$; nó đổi lấy cái gì, và khi nào phép đổi đó không đáng?

Nguồn

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.

1.6 Thực hành

Chương này khác các chương khác ở một điểm: đọc hiểu không tạo ra kỹ năng. Hai mục dưới đây là phần đọc và phần tự vấn; phần tay chân nằm ở khối mini project cuối chương, và đó mới là chỗ kiến thức chuyển thành phản xạ.

Repo và tài nguyên. Để hiểu autodiff từ gốc, [karpathy/micrograd](#) là bản cài đặt reverse-mode gọn trong khoảng một trăm dòng và đọc hết trong một buổi; [tinygrad/tinygrad](#) là bước tiếp theo, đủ nhỏ để đọc nhưng đã có đầy đủ khái niệm của một framework thật. Để tự derive gradient theo lối bài tập, bộ assignment của CS231n vẫn là tài liệu tốt nhất, và [labmlai/annotated_deep_learning_paper_implementations](#) cho bạn bản cài đặt có chú giải của phần lớn kỹ thuật nêu trong chương. Về augmentation, [alumentations-team/alumentations](#) và `transforms.v2` là hai lựa chọn mặc định. Về huấn luyện phân tán, đọc trực tiếp tài liệu DDP và FSDP của PyTorch trước, rồi mới tới [microsoft/DeepSpeed](#) nếu cần các giai đoạn ZeRO sâu hơn. Danh sách này chốt tại tháng 9/2026, và tên các mô hình dẫn đầu sẽ cũ trong sáu tới mười hai tháng. Nhưng phần lớn nội dung chương này là hạ tầng chứ không phải mô hình, nên nó bền hơn hẳn mức trung bình của cây.

Câu hỏi kiểm tra hiểu. Vì sao AdamW khác Adam cộng L2, và bạn kỳ vọng khác biệt đó lớn cỡ nào trong thực tế? Một đường cong học giảm rất nhanh rồi phẳng ở mức cao — nêu ba nguyên nhân khả dĩ và ba phép đo phân biệt chúng. Vì sao BatchNorm làm lượt ngược tốn thêm bộ nhớ, và điều đó gợi ý gì khi bạn phải chọn giữa BatchNorm và GroupNorm ở một bài detection chạy hai ảnh mỗi GPU?

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Lan truyền ngược	Thuật toán tính gradient bằng cách đi ngược đồ thị phép tính đúng một lượt. Nó rẻ hơn vì phân thuận khoảng tám bậc độ lớn, vì một lượt trả về đạo hàm theo <i>mọi</i> tham số. Nó không bao giờ dừng ma trận Jacobian, chỉ áp tích vector-Jacobian ở từng nút.
•• Lượt xuôi, lượt ngược	Lượt xuôi tính giá trị và ghi lại kết quả trung gian; lượt ngược dùng chúng để tính gradient. Cả hai là <i>một lần chạy</i> của lan truyền ngược, dừng nhằm với chính thuật toán.
••• Ba mẫu gradient	Cộng phân phối adjoint đều cho mọi nhánh. Nhân hoán đổi: mỗi nhánh nhận adjoint nhân giá trị nhánh kia. Max định tuyến toàn bộ về nhánh thắng. Mẫu thứ tư hay bị quên: rẽ nhánh thì cộng dồn, nên lượt ngược của broadcast là một phép cộng theo trục.
••• Bộ nhớ kích hoạt	Toàn bộ đầu ra trung gian của lượt xuôi, phải giữ lại vì lượt ngược cần tới. Nó đạt đỉnh ở chỗ hai lượt giao nhau, nên nó mới là thứ chặn batch chứ không phải số trọng số. Gradient checkpointing hạ đỉnh xuống $O(\sqrt{L})$ với giá khoảng 30 phần trăm thời gian.
••• Vì sao gradient tắt hoặc nổ khi mạng sâu?	Vì adjoint bị nhân dồn qua từng tầng. Nếu hệ số mỗi tầng là ρ thì gradient ở tầng đầu tỉ lệ với ρ^L ; $\rho = 0,9$ qua 50 tầng còn $5 \cdot 10^{-3}$.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Khởi tạo He	Đặt $\text{Var}(w) = 2/n_{\text{in}}$ để phương sai tín hiệu không đổi qua mỗi tầng. Hệ số 2 đến từ việc ReLU xoá một nửa số đầu ra, nên nó đúng cho ReLU chứ không đúng cho tanh.
●●● BatchNorm	Chuẩn hoá kích hoạt theo thống kê của cả batch, rồi trả quyền chỉnh lại cho mạng qua γ và β . Vì nó ghép các mẫu trong batch lại với nhau, nó hỏng khi batch nhỏ, khi train và eval dùng hai nguồn thống kê, và khi các GPU không đồng bộ.
●● Val loss thấp bất thường, và đổi khi đổi batch size lúc đánh giá — nguyên nhân?	Quên gọi <code>model.eval()</code> , nên BatchNorm dùng thống kê của chính batch kiểm thử. Phân biệt với rò rỉ dữ liệu: rò rỉ cho điểm cao ổn định, còn lỗi này đổi điểm khi bạn đổi batch size lúc đánh giá.
●●● Vì sao momentum thắng trong rãnh hẹp?	Thành phần vuông góc với rãnh đổi dấu mỗi bước nên triệt tiêu trong tổng trượt. Thành phần dọc rãnh giữ nguyên dấu nên cộng dồn tới hệ số $1/(1 - \mu)$, gấp 10 lần khi $\mu = 0.9$.
●●● AdamW	Adam với weight decay tách khỏi mẫu số thích nghi, cộng thẳng vào bước cập nhật. Nhờ đó suy giảm là một phép co đều, không phụ thuộc lịch sử gradient. <code>Adam(weight_decay=...)</code> không phải AdamW.
●●● Vì sao learning rate là siêu tham số quan trọng nhất?	Vì nó là đại lượng duy nhất nhân trực tiếp vào mọi bước cập nhật. Sai một bậc ở nó đổi hẳn bản chất quá trình; sai một bậc ở weight decay chỉ mất vài phần trăm điểm số. Warmup tồn tại cũng vì lý do này: $\hat{v}$ của Adam ở vài trăm bước đầu quá nhiều để tin.
●● Vì sao fp16 làm huấn luyện tê liệt?	Gradient thị giác thường nằm quanh 10^{-7} tới 10^{-4} , dưới ngưỡng chuẩn hoá $6,1 \cdot 10^{-5}$ của fp16, nên bị làm tròn về 0. Loss scaling nhân loss với 2^{16} để dịch cả phổ vào vùng biểu diễn được, rồi chia lại trước bước cập nhật.
●●● Loss thành NaN trong vài chục bước đầu — nguyên nhân?	Learning rate quá lớn, hoặc tràn số ở fp16. Phân biệt: hạ learning rate mười lần mà vẫn NaN thì là tràn số, hãy thử bf16 hoặc bật loss scaling.
●●● Loss giảm gần tuyến tính và không bão hoà khi hết lịch — nguyên nhân?	Learning rate quá nhỏ. Phân biệt với mô hình thiếu sức chứa: đo tỉ lệ cập nhật trên trọng số, dưới 10^{-5} là learning rate, còn quanh 10^{-3} mà loss vẫn cao thì là sức chứa hoặc dữ liệu.
●● Loss răng cưa với chu kỳ đúng bằng một epoch — nguyên nhân?	Quên shuffle, nên mô hình gặp lại đúng thứ tự dữ liệu mỗi vòng. Phân biệt với learning rate quá cao: vẽ loss theo từng bước, răng cưa có chu kỳ đều là shuffle, còn nhiễu không chu kỳ là learning rate.
●●● Train loss giảm bình thường nhưng val accuracy quanh mức ngẫu nhiên — nguyên nhân?	Nhân bị lệch chỉ số hoặc ghép sai mẫu: mạng học thuộc một ánh xạ sai nhưng nhất quán. Phân biệt bằng ma trận nhầm lẫn — nó lệch hẳn khỏi đường chéo đúng một cột.
●● Augmentation	Sinh thêm mẫu bằng cách biến đổi mẫu có sẵn. Mỗi phép là một tuyên bố “nhân không đổi dưới phép này”, nên nó thêm thông tin chứ không ràng buộc sức chứa. Nó hỏng khi phép biến đổi phá nhân, ví dụ lật ngang một biển báo rẽ.
●● ZeRO và FSDP	Chia trọng số, gradient và trạng thái optimizer giữa các tiến trình, theo ba giai đoạn tăng dần. Nó là hệ quả trực tiếp của bảng kê toán $16P$ byte, và trả giá bằng lưu lượng giao tiếp.
●●● Overfit một batch nhỏ	Huấn luyện 20–50 mẫu tới loss gần 0 sau khi tắt mọi augmentation. Không đạt thì bug nằm trong vòng lặp chứ không ở khả năng tổng quát hoá. Nó vẫn qua nếu nhân bị hoán vị theo một quy tắc cố định.
● Double descent	Vượt qua ngưỡng nội suy, sai số kiểm thử đạt đỉnh rồi <i>giảm trở lại</i> khi mô hình lớn thêm. Hệ quả: trực giác chữ U của giáo trình cũ không dùng được cho mạng quá tham số.

●●● phải thuộc ●● nên nắm ● biết là có

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
PyTorch autograd, JAX	Toàn bộ cơ chế tính gradient là hiện thực của vi phân ngược trên đồ thị: mỗi phép toán khai báo một tích vector-Jacobian	Hạ tầng bên; API gần như không đổi từ 2017, nên đây là phần ít mất giá nhất của chương
FlashAttention	Tính lại theo ô thay vì lưu ma trận $T \times T$ — chính là ý tưởng gradient checkpointing áp cho attention, đổi phép tính lấy bộ nhớ	Mặc định trong phần lớn thư viện transformer hiện nay
DeepSpeed ZeRO, PyTorch FSDP	Hệ quả trực tiếp của bảng kế toán $16P$ byte: chia trọng số, gradient và trạng thái Adam giữa các tiến trình	Ba giai đoạn ZeRO tương ứng ba dòng của bảng đó
AdamW làm mặc định trong <code>timm</code> , <code>transformers</code> , và phần lớn recipe thị giác	Tách weight decay khỏi mẫu số thích nghi — đúng bước cuối của mạch bày optimizer	Lỗi Adam cộng L2 vẫn còn trong nhiều mã nguồn cũ
<code>torch.amp</code> , bf16 trên GPU thế hệ Ampere trở đi	Loss scaling và master weights fp32 — hai hệ quả trực tiếp của phân tích dải động	Bf16 làm loss scaling gần như không cần, nhưng không bỏ được master weights
RandAugment, Mixup và CutMix trong <code>timm</code> và <code>transforms.v2</code>	Augmentation như nơi phát biểu bất biến, và như công cụ chính quy hoá mạnh nhất trong thị giác	Chính các thành phần recipe này là thứ ConvNeXt dùng để bắt kịp ViT ở chương 13

MINI PROJECT SLAM · Bảng triệu chứng cho mạng chấm điểm ghép đặc trưng · 1-2 ngày

Mục tiêu	tạo một thư viện triệu chứng dùng được suốt sự nghiệp: huấn luyện một mạng nhỏ dự đoán độ tin cậy của một cặp ghép đặc trưng ORB, rồi cố ý phá nó tám cách để học nhận ra từng dạng hỏng <i>chỉ bằng hình dạng đường loss</i> , trước khi cần tới chúng trong một hệ thật.
Dữ liệu	một chuỗi EuRoC bất kỳ, chẳng hạn MH_01. Trích cặp ghép ORB giữa các khung hình liên tiếp; gán nhãn dương khi sai số tái chiếu dưới ngưỡng theo chân trị pose, âm khi vượt. Vài chục nghìn cặp là quá đủ; đầu vào có thể chỉ là khoảng cách Hamming, chênh lệch góc, chênh lệch tỉ lệ và toạ độ chuẩn hoá.
Các bước	chạy một lần baseline tới khi loss xác thực phẳng và ghi lại ba chỉ báo sức khoẻ; rồi chạy tám phiên bản hỏng có chủ đích — learning rate nhân 100, learning rate chia 100, quên shuffle, quên eval(), nhãn lệch một chỉ số, chuẩn hoá đầu vào bằng thống kê của chuỗi khác, khởi tạo toàn hằng số, tắt chuẩn hoá giữa mạng; vẽ cả chín đường loss vào <i>một</i> hình; cuối cùng che nhãn và tự đoán từng đường là lỗi nào.
Xong khi	bạn có bảng ánh xạ chín đường cong sang chín nguyên nhân, nhận đúng ít nhất bảy trên chín khi che nhãn, và với mỗi cặp dễ nhằm nêu được một phép đo phân biệt — tỉ lệ cập nhật trên trọng số, hoặc mức đổi điểm số khi đổi batch size lúc đánh giá.
Nối với	C/12/04 cho cây chẩn đoán mà bảng này bổ sung số liệu; C/12/03 cho ba chỉ báo sức khoẻ dùng làm phép đo phân biệt; tập cặp ghép đã gán nhãn tạo ở đây dùng dùng lại nguyên vẹn cho mini project của chương 13.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- Vì vi phân ngược trả về đạo hàm theo *mọi* đầu vào chỉ trong một lượt duy nhất, còn vi phân thuận cho đạo hàm theo đúng một hướng mỗi lượt. Bài toán huấn luyện có một đầu ra và 10^9 đầu vào, tức đúng hình dạng mà vi phân ngược phục vụ; tỉ số chi phí giữa hai cách chính là tỉ số giữa số đầu vào và số đầu ra.
- Vì quy tắc chuỗi cho ra một *tích* của L Jacobian, và tích lũy thừa thì hoặc tắt hoặc nổ — một hệ số 0,9 qua 50 tầng còn $5 \cdot 10^{-3}$. Cái thiếu suốt hai mươi năm không phải công thức đạo hàm mà là ba cách giữ hệ số đó gần 1: hàm kích hoạt không bão hoà, khởi tạo theo lập luận phương sai, và chuẩn hoá.

- A3. Vì chúng nằm trên hai nhánh giải quyết hai vấn đề khác nhau — momentum lọc *hướng*, Ada-Grad và RMSProp chỉnh *độ dài bước* cho từng tham số — rồi hợp lại ở Adam. AdamW là cái sinh ra chỉ để sửa lỗi của phép hợp nhất đó: khi hình phạt L2 bị chia cho mẫu số thích nghi, nó không còn là weight decay nữa.
- A4. Vì nghiệm không do lớp hàm quyết định một mình, mà do *quá trình* tìm nghiệm quyết định. Nhiều của SGD, khởi tạo và lịch học đều thiên vị những nghiệm nhất định. Ngành gọi đó là chính quy hoá ngầm và chưa mô tả được nó một cách định lượng — nên các cận sai số dựa trên sức chứa gần như rỗng.
- A5. Vì dải động và độ chính xác là hai chuyện khác nhau. bf16 chỉ có 7 bit định trị, tức độ chính xác tương đối khoảng $4 \cdot 10^{-3}$. Bước cập nhật lành mạnh lại chỉ bằng 10^{-3} độ lớn trọng số. Cộng một số nhỏ như vậy vào trọng số bf16 thì phép cộng làm tròn về giá trị cũ, và bước đi biến mất hoàn toàn.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn phải chỉnh một cây đàn piano khổng lồ có mười triệu sợi dây, và mỗi lần chỉnh xong bạn chỉ được nghe đúng một hợp âm rồi nhận đúng một con số: nghe chênh bao nhiêu. Nếu thử từng dây một — vặn nhẹ, gảy lại, nghe xem con số đổi thế nào — thì bạn cần mười triệu lần gảy chỉ để biết nên vặn theo hướng nào. Không ai làm nổi.

Mẹo của cả chương là đừng đi xuôi mà đi ngược. Sau khi gảy một lần, bạn bắt đầu từ tiếng chênh ở đầu ra và lần ngược qua từng cơ cấu truyền lực, hỏi mỗi cái đúng một câu rất hẹp: kéo đầu ra của anh nhích một li thì đầu vào của anh phải nhích bao nhiêu. Mỗi cơ cấu chỉ cần biết quy đổi nội bộ của riêng nó, và một vòng đi ngược duy nhất trả lời cho cả mười triệu dây.

Cái giá có ba phần. Thứ nhất, muốn đi ngược được thì lượt đi xuôi phải ghi nhớ trạng thái của mọi cơ cấu, nên thứ chận bạn thường là chỗ để ghi chứ không phải thời gian. Thứ hai, lực truyền qua ba mươi khớp nối: mỗi khớp nuốt một chút thì tới cuối chẳng còn gì, mỗi khớp thêm một chút thì nó giật đứt dây — gần như mọi thủ thuật còn lại chỉ để giữ mỗi khớp truyền đúng bằng lực nhận vào. Thứ ba, vặn bao nhiêu mỗi lần: mạnh tay thì vọt qua, nhẹ tay thì cả đời không xong.

Chỗ không rút gọn được: vì sao một cách chỉnh nằm ở vùng “phẳng” — vùng mà vặn lệch đi một chút vẫn nghe được — lại chơi hay hơn trên những bản nhạc chưa từng thử, thì tới nay chưa có lời giải thích vừa gọn vừa đúng. Mọi cách nói đơn giản hiện có đều bỏ sót một phần ví dụ, nên đây là chỗ nên biết là mình chưa biết.

Một câu: Đi xuôi một lần để ghi nhớ, đi ngược một lần để quy trách nhiệm cho từng núm vặn, rồi vặn thật nhẹ — và mọi thứ còn lại chỉ là giữ cho lực truyền qua ba mươi khớp nối không tắt cũng không giật.

1.7 Kết nối

Chương này đứng ở giữa cây và có hai chiều nối rõ ràng. Ngược lên, nó tiêu thụ A/01-toan-toi-thieu cho quy tắc chuỗi và lập luận phương sai, và B/09-thiet-ke-ham-mat-mat cho thứ mà toàn bộ bộ máy này đang tối thiểu hoá. Xuôi xuống, nó là tiền đề của *mọi* chương còn lại trong Phần C: không có nó thì các kiến trúc ở chương 13 tới 18 chỉ là sơ đồ khối không chạy được. Hai nhánh nối ngang đáng đi ngay sau chương này. Thứ nhất là B/11-gioi-han-tinh-toan: ở đó bộ nhớ kích hoạt và dải động số được đặt vào mô hình roofline. Thứ hai là E/24-thi-nghiem-va-ablation: ở đó bạn học cách không để một bảng so sánh dẫn tới kết luận sai, vì bảng ấy thường chỉ phản ánh khác biệt công thức huấn luyện.

Tổng kết chương

Ta đã đi từ một phép nhân Jacobian cục bộ tới việc chia trạng thái optimizer giữa nhiều GPU, và điều đáng giữ lại là toàn bộ hành trình đó có một sợi chỉ duy nhất. Mục 1 cho thấy vì sao đạo hàm rẽ và cái giá của nó là bộ nhớ. Mục 2 cho thấy tích các Jacobian là đại lượng phải kiểm soát,

và ba nhóm kỹ thuật tưởng rời rạc đều nhắm vào nó. Mục 3 cho thấy bảy optimizer là bảy lần vá liên tiếp, và learning rate vẫn là siêu tham số không có đối thủ. Mục 4 cho thấy phần lớn các lần “mô hình không tổng quát hoá” thực ra là lỗi trong vòng lặp, và đưa ra quy trình tách hai thứ đó.

Còn bỏ ngỏ hai chỗ lớn. Thứ nhất là chính quy hoá ngầm: ta chưa biết vì sao SGD lại chọn được nghiệm tổng quát hoá tốt trong một không gian đầy nghiệm học thuộc, nên mọi kết luận về sức chứa mô hình đều nên phát biểu thận trọng. Thứ hai là chương này cố tình bỏ qua kết nối tất, thành viên thứ tư của họ “giữ tích Jacobian gần 1”, vì nó là một lựa chọn kiến trúc; nó mở đầu chương tiếp theo. Đọc tiếp C/13-kien-truc-backbone với một câu hỏi trong đầu: trong mỗi bước tiến hoá kiến trúc, bao nhiêu phần là kiến trúc thật và bao nhiêu phần là công thức huấn luyện vừa học ở đây?

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Sợi chỉ của cả chương là giữ tích các Jacobian không chết cũng không nổ. Khởi tạo, normalization và lịch learning rate tấn công tích đó ở ba chỗ khác nhau nào, và vì sao có cả ba vẫn chưa đủ để bỏ warmup?
2. BatchNorm xuất hiện ở cả bốn mục với bốn vai khác nhau: gradient của một mẫu phụ thuộc cả batch, ảnh hưởng tới số điều kiện, tương tác với weight decay, và giới hạn của gradient accumulation. Bốn điều đó suy ra từ đúng một tính chất — tính chất nào, và nó ép bạn chọn gì khi huấn luyện phân tán?
3. Bộ nhớ kích hoạt của mục 1, trạng thái optimizer của mục 3 và phân mảnh ZeRO của mục 4 là ba khoản chi bộ nhớ khác nhau. Khi một lần chạy hết bộ nhớ, bạn cắt khoản nào trước, và mỗi lựa chọn đổi lấy cái gì?
4. Loss huấn luyện phẳng ở mức cao có thể do hình dạng landscape, do công thức huấn luyện, hoặc do một lỗi trong vòng lặp dữ liệu. Ba phép thử nào tách ba nguyên nhân đó, và vì sao phép “overfit một batch” phải chạy trước hai phép còn lại?
5. Vì sao một cải tiến đo trong công thức huấn luyện cũ có thể biến mất khi đo lại trong công thức mới, và điều đó ràng buộc thể nào cách bạn thiết kế ablation cho những kỹ thuật trong chương này?
6. Mục 4 thừa nhận ta chưa biết vì sao SGD chọn được nghiệm tổng quát hoá tốt. Với khoảng trống đó, quyết định thực hành nào trong chương vẫn giữ nguyên căn cứ, và quyết định nào chỉ còn là kinh nghiệm?
7. Bạn hạ mixed precision từ fp32 xuống bf16 và loss xấu đi một chút. Ba nguyên nhân nằm ở ba mục khác nhau của chương là gì, và mỗi nguyên nhân phân biệt bằng phép đo nào?

Nguồn

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [2] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [3] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *AISTATS*, 2010.
- [4] K. He, X. Zhang, S. Ren, et al., “Delving Deep into Rectifiers,” in *ICCV*, 2015.
- [5] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [7] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [8] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [10] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [11] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *ICLR*, 2017.
- [12] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [13] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [14] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [15] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [16] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [17] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [18] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [19] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [22] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [23] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.

- [24] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.